FACULTY OF ENGINEERING, COMPUTING AND THE ENVIRONMENT

School of Computing and Mathematics

MSc DEGREE

IN

DATA SCIENCE

Evans Konadu

K2436512

AI-Powered Pneumonia Detection in Low Resource Settings

Experiment-Based Dissertation

September, 2025

Supervisor: Prof. Dimitrios Makris

Kingston University London

Table of Contents

List of Figures i
List of Tables ii
Acknowledgements iii
Abstract iv
1. Introduction 1
1.1 Background and Motivation 1
1.2 Aim and Objectives 1
2. Literature Review 2
3. Methodology 4
3.1 Research Design and Overview 4
3.2 Datasets and Preprocessing Pipeline 5
3.2.1 Dataset Selection and Rationale 5
3.2.2 Data Splitting and Class Balancing Strategies 7
3.3 Model Architecture Design and Implementation 8
3.3.1 Individual Model Architectures 8
3.3.2 Novel Architecture Contributions 8
3.3.3 Quantisation and Optimisation Methodology 10
3.4 Performance Evaluation Framework 10
3.4.1 Evaluation Metrics 10
3.4.2 Explainability Integration 11
4. Implementation 11
4.1 Software Architecture and Development Environment 11
4.2 Model Training Implementation 11
4.3 Implementation Challenges 11
4.4 Quantisation and Edge Deployment 12
4.4.1 TensorFlow Lite Implementation 12
4.4.2 Edge Deployment Simulation 13
4.5 Explainability Implementation 14
4.6 Data Management and Ethics 14
5. Experiments, Results and Discussion 14
5.1 Data Preprocessing and Augmentation Implementation 14
5.2 Training Configuration and Performance Analysis 14
5.3 Ensemble Architecture Analysis 16
5.4 Comparative Performance Analysis Across Datasets 17
5.5 Model Optimisation Through Quantisation 18
5.6 Edge Deployment Performance Evaluation 19
5.7 Explainability Integration and Performance 20
5.8 Optimal Architecture Selection and Clinical Deployment 21

6. Conclusions and Future Work 22
References 23
Appendix 26

List of Figures

Figure 3.1: Methodology Flowchart showing the complete workflow 4
Figure 3.2: Distribution of 14 Labels in NIH ChestX-ray14 Dataset - Bar chart with annotated counts and pie chart with percentage annotations showing all 14 labels..... 5
Figure 3.3: Class Distribution for Pneumonia Detection Task - Bar chart and pie chart highlighting the imbalance (60,412 Normal vs. 1,431 Pneumonia) 5
Figure 3.4: Distribution of Labeled Chest X-Ray Images (Mendeley Data) - Bar chart with counts and pie chart with percentages for Normal and Pneumonia classes..... 6
Figure 3.5: Sample X-Ray Images from Mendeley Dataset..... 6
Figure 3.6: Sample X-Ray Images from NIH ChestX-ray14 Dataset 6
Figure 3.7: Class Distribution Across Train, Validation, and Test Sets - NIH Dataset..... 7
Figure 3.8: Class Distribution Across Train, Validation, and Test Sets - Mendeley Dataset..... 7
Figure 3.9: SE-MobileNetV2 Architecture..... 9
Figure 3.10: SE-EfficientNetB3 Architecture..... 9
Figure 3.11: PneumoLiteCovNet-25 Architecture..... 9
Figure 3.12: Nuclear Ensemble Architecture 10
Figure 4.1: SE-Block integration code 12
Figure 4 2: Nuclear Ensemble dual input handling code..... 12
Figure 4.3: Multi-scale feature extraction code..... 12
Figure 4.4: Edge Deployment Workflow..... 13
Figure 5.1: Geometric Data Augmentation Examples - Demonstrating rotation, zoom, shift, and shear transformations applied whilst preserving anatomical orientation..... 14
Figure 5.2: Comparative AUROC Performance Across Datasets with CheXNet Benchmark - Showing performance differential between paediatric and adult imaging 18
Figure 5.3: Quantisation Impact Analysis - Comparing original versus quantised model performance across key metrics 19
Figure 5.4: Grad-CAM Attention Visualisations - Xception (Mendeley Dataset) - Demonstrating model attention regions for pneumonia detection in paediatric chest X-rays 20
Figure 5.5: Grad-CAM Attention Visualisations - EfficientNetB4 (NIH Dataset) - Showing interpretability maps for adult chest X-ray analysis..... 20
Figure 5.6: Confusion Matrix and ROC Analysis - Xception (Mendeley Dataset)..... 21
Figure 5.7: Confusion Matrix and ROC Analysis - EfficientNetB4 (NIH Dataset)..... 21

List of Tables

Table 1: Dataset Characteristics and Distribution 6
Table 2: Model Architecture Specification 8
Table 3: Computational Platform Specifications for Edge Deployment Evaluation 13
Table 4: NIH ChestX-ray14 Dataset Training and Evaluation Performance Summary (Metrics at Best Epoch) 15
Table 5: Mendeley Dataset Training and Evaluation Performance Summary (Metrics at Best Epoch) 15
Table 6: Nuclear Ensemble Component Training Performance Summary (Metrics at Best Epoch) 16
Table 7: Nuclear Ensemble Component Performance Comparison - Showing individual and combined performance across architectures 16
Table 8: Comparative Performance Metrics Across Datasets (Pre-Quantisation) - Showing performance variations between paediatric and adult populations 17
Table 9: Original vs Quantised Model Performance Comparison - Demonstrating size reduction and performance preservation across architectures 18
Table 10: Edge Deployment Inference Performance - Showing inference times on Raspberry Pi-simulated environments 19

Acknowledgements

I am deeply grateful to God for His guidance, strength, and protection throughout this journey.

I extend my heartfelt appreciation to my supervisor, Professor Dimitrios Makris, whose unwavering encouragement and technical expertise guided me through every phase of this work. His support in exploring new avenues and following my instincts, combined with his ability to redirect me when obstacles arose, has been instrumental in expanding my understanding and introducing me to new concepts and ideas.

My sincere thanks go to Dr Regina Ofori-Boateng, whose friendship and encouragement instilled in me the hope and confidence that this achievement was possible.

I am profoundly grateful to my parents and siblings for their constant prayers and unwavering support. Special appreciation extends to the Nyarkoh family, the Bilson family, and Revd. Capt. Kingsley Yeboah and his family for their unflinching support throughout this endeavour.

Their collective belief in my abilities has been the foundation upon which this work stands.

Abstract

Pneumonia remains the leading infectious cause of mortality among children under five, claiming over 700,000 lives annually, with diagnostic challenges compounded by radiologist shortages in resource-constrained healthcare settings. This research addresses the critical gap between advanced deep learning pneumonia detection capabilities and practical deployment limitations in low-resource clinical environments through systematic development and optimisation of multiple architectures for automated chest X-ray pneumonia detection targeting edge device deployment. Ten architectures were comprehensively evaluated across paediatric Mendeley (5,856 images) and adult NIH ChestX-ray14 (112,120 images) datasets, including novel contributions: SE-MobileNetV2, SE-EfficientNetB3, PneumoLiteCovNet-25, and Nuclear Ensemble. Systematic quantisation achieved up to 91% model size reduction whilst preserving clinical utility, with optimised Grad-CAM enabling real-time interpretability. Results identified optimal architectures: Xception achieved 97.12% accuracy on paediatric data (21.61 MB, 550.76 ms inference), whilst EfficientNetB4 demonstrated 80.94% accuracy on adult imagery (18.59 MB, 232.69 ms inference), substantially exceeding CheXNet baseline (0.8986 vs 0.7679 AUROC). This integrated framework demonstrates clinically acceptable automated pneumonia detection within severe computational constraints, providing foundation for democratised healthcare delivery in low-resource settings where pneumonia burden is highest.

1. Introduction

1.1 Background and Motivation

Pneumonia remains the leading infectious cause of mortality among children under five years, claiming over 700,000 lives annually and accounting for one death every 43 seconds [1]. This burden disproportionately affects South Asia and sub-Saharan Africa, where healthcare infrastructure limitations compound diagnostic challenges [1]. Despite global health initiatives, progress in reducing pneumonia mortality has lagged behind improvements for other infectious diseases, highlighting critical gaps in current diagnostic approaches.

Accurate diagnosis relies predominantly on chest X-ray interpretation, yet radiologist shortages in resource-constrained settings create dangerous delays in treatment initiation [2]–[6]. These environments, characterised by limited computational resources, unreliable power supplies, and minimal technical infrastructure, struggle to implement conventional diagnostic protocols effectively [14]. The emergence of deep learning has demonstrated remarkable potential for automated pneumonia detection, with models achieving radiologist-level performance on benchmark datasets [7], [8]. However, these achievements remain largely theoretical for communities most affected by pneumonia, as state-of-the-art models require computational resources unavailable in rural clinics.

The fundamental challenge lies not in diagnostic accuracy but in practical deployment. Contemporary deep learning models demand substantial memory, processing power, and stable electricity supplies that exceed the capabilities of basic healthcare facilities [9], [10], [13]. Modern architectures often require graphics processing units, gigabytes of RAM, and consistent power infrastructure that simply do not exist in settings where pneumonia burden is highest. This creates a troubling paradox where technological advances fail to reach populations with the highest disease burden, widening rather than narrowing global health inequities. Supporting Sustainable Development Goal 3.2.1 requires solutions that balance diagnostic performance with computational efficiency, enabling meaningful deployment where needed most whilst maintaining the accuracy standards necessary for clinical decision-making [11].

This project addresses these challenges through systematic development and optimisation of deep learning models specifically designed for resource-constrained deployment. By evaluating multiple architectures across diverse datasets, implementing advanced quantisation techniques, and validating performance on edge hardware, this research aims to bridge the gap between AI innovation and practical healthcare delivery. The methodology encompasses training models on both the Kermany paediatric dataset and the NIH ChestX-ray14 dataset, enabling robust multi-dataset validation essential for real-world generalisation. The integration of explainability mechanisms through Grad-CAM visualisation further ensures clinical acceptance, creating a comprehensive framework for deployable pneumonia detection in low-resource settings that maintains diagnostic reliability whilst operating within severe computational constraints.

The remainder of this dissertation is organised as follows: Section 2 provides a critical review of the literature and state-of-the-art in automated pneumonia detection. Section 3 describes the datasets and experimental methodology. Section 4 details the implementation approach. Section 5 presents the experiments, results and discussion, including comprehensive performance and deployment evaluations. Finally, Section 6 concludes with key findings and outlines future directions for research and practical deployment.

1.2 Aim and Objectives

Aim: To develop a computationally efficient and interpretable deep learning model for pneumonia detection in chest X-rays, optimised for deployment in low-resource clinical settings.

Objectives:

1. Conduct a comprehensive review to identify deep learning architectures and optimisation techniques suitable for edge devices.
2. Pre-process and balance chest X-ray datasets to ensure robust training across paediatric and adult populations.
3. Develop and train multiple architectures including lightweight models, enhanced variants, and ensemble methods.
4. Implement quantisation techniques to optimise models for low-power hardware deployment.
5. Assess diagnostic performance and deployment feasibility through simulation on edge devices.
6. Integrate Grad-CAM visualisation to enhance clinical interpretability and physician trust.
7. Compare optimal architectures against established standards including CheXNet baseline performance.

2. Literature Review

Deep learning has transformed medical imaging analysis, particularly in pneumonia detection, where convolutional neural networks now rival or exceed human radiologist performance. Landmark studies have established the technical feasibility of automated diagnosis, yet translation to clinical practice remains limited, especially in resource-constrained healthcare settings where pneumonia burden is highest. The disconnect between laboratory achievements and clinical implementation reflects fundamental misalignment between model development priorities and deployment realities. This review examines current approaches across three critical dimensions: diagnostic accuracy, computational efficiency, and clinical interpretability, whilst identifying pathways toward practical solutions for underserved populations.

Contemporary pneumonia detection models demonstrate impressive performance on curated datasets, often surpassing human radiologist benchmarks. Kermany et al. achieved 92.8% accuracy with transfer learning on paediatric chest X-rays, establishing foundational benchmarks for subsequent research [12]. More significantly, CheXNet's 121-layer DenseNet architecture achieved an F1 score of 0.435 compared to radiologist average of 0.387, demonstrating that automated systems could exceed human performance even within complex multi-pathology frameworks [8]. Ensemble approaches have pushed boundaries further, with Kundu et al. reporting 98.81% accuracy combining GoogLeNet, ResNet-18, and DenseNet-121 on the Kermany dataset [17], and Chouhan et al. achieving 96.4% accuracy with 99.0% sensitivity using multi-model ensembles [27]. These results suggest deep learning has solved the technical challenge of pneumonia detection from a pure performance perspective. However, such conclusions overlook fundamental deployment constraints.

The computational demands of high-performing models create significant barriers to practical implementation. CheXNet, whilst achieving superior performance to radiologists, requires a 121-layer DenseNet architecture with over 8 million parameters [8]. Though this represents strong performance for a model designed to detect 14 different pathologies simultaneously, its computational requirements far exceed the capabilities of basic healthcare facilities operating with minimal computing resources. The model demands dedicated GPU processing, substantial RAM allocation, and stable power supply—infrastructure absent in settings where pneumonia mortality is highest. This exemplifies the broader challenge where architectural complexity necessary for high accuracy conflicts with deployment feasibility in settings lacking reliable electricity or modern computing infrastructure [9]. The irony is stark: populations with the greatest need for automated diagnostic support are precisely those unable to deploy existing solutions.

Recent research has attempted to address these constraints through lightweight architectures. MobileNet variants have shown particular promise, with Al Reshan demonstrating 94.23% accuracy on pneumonia detection tasks [28]. These efficiency-focused approaches reduce computational demands substantially, yet often sacrifice explainability mechanisms essential for clinical acceptance. As highlighted in systematic reviews, the ability to visualise and understand model decisions through techniques like attention mapping significantly impacts physician trust and adoption [29]. The trade-off between efficiency and interpretability remains largely unresolved in current literature.

Model optimisation techniques, particularly quantisation, offer potential pathways for deployment-ready solutions without requiring architectural redesign. Thakur et al. demonstrated that post-training quantisation could achieve 80% model size reduction with 32-fold inference speed improvements whilst maintaining diagnostic utility, converting 32-bit floating-point weights to 8-bit integers [13]. This approach proves particularly valuable as it enables retrofitting existing high-performance models for edge deployment. However, medical imaging presents unique challenges for precision reduction. Subtle radiographic features indicative of early pneumonia may be lost through aggressive quantisation, potentially compromising diagnostic sensitivity. Current studies rarely validate quantised models on actual edge hardware or provide comprehensive deployment metrics necessary for implementation decisions.

The fragmentation in existing research reflects methodological limitations that hinder practical progress. Studies typically excel in single dimensions, achieving either exceptional accuracy through complex architectures, computational efficiency through lightweight designs, or interpretability through attention mechanisms, but rarely address these requirements holistically [25], [26]. Multi-dataset validation remains limited, with most research focusing on single datasets despite significant variations in imaging equipment, patient demographics, and disease presentation across clinical settings. This raises questions about model generalisability essential for real-world deployment.

Infrastructure analyses by Agbeyangi and Suleman emphasise that viable solutions must accommodate devices with limited processing power, constrained memory, and unreliable connectivity [14]. Yet current literature provides minimal evidence of systematic edge deployment validation. Studies reporting impressive accuracy metrics typically omit inference time measurements on representative hardware platforms like Raspberry Pi devices commonly available in low-resource settings [22], [23]. This gap between laboratory performance and practical viability represents a critical barrier to clinical implementation.

Recent attention-based approaches illustrate both progress and persistent challenges. Wang et al.'s PneumoFusion-Net achieves precise localisation through sophisticated attention mechanisms [2], whilst An et al.'s ensemble demonstrates 98.38% precision with interpretable outputs [7]. However, these advances come within frameworks exceeding 300MB model sizes, rendering them unsuitable for edge deployment. The continued pursuit of marginal accuracy improvements through architectural complexity overlooks the fundamental requirement for deployable solutions in resource-constrained environments.

The path forward requires integrated approaches that balance competing demands through systematic optimisation across multiple dimensions. Systematic reviews emphasise the need for comprehensive methodologies addressing accuracy, efficiency, and interpretability simultaneously rather than treating them as mutually exclusive objectives [15], [29]. This includes comprehensive dataset evaluation to ensure generalisability, quantisation strategies validated on target hardware platforms, and explainability mechanisms designed for clinical workflows. Only through such holistic approaches can deep learning fulfil its promise of democratising quality healthcare, particularly for communities where pneumonia burden remains highest and resources most limited.

3. Methodology

This section presents the systematic approach employed to develop, optimise, and evaluate deep learning models for pneumonia detection in chest X-rays, targeting deployment in low-resource clinical settings.

3.1 Research Design and Overview

The experimental design follows a comparative evaluation framework wherein multiple deep learning architectures are systematically trained and assessed on two distinct chest X-ray datasets. Novel architectural innovations focus on improving efficiency and adaptability whilst achieving outstanding results on complex cognitive tasks. Ensemble learning techniques leverage diverse learners for comprehensive pattern recognition, whilst quantisation converts floating-point networks into low-bit-width integer networks for efficient edge deployment.

Figure 3.1 illustrates the complete methodological workflow from dataset acquisition to final evaluation.

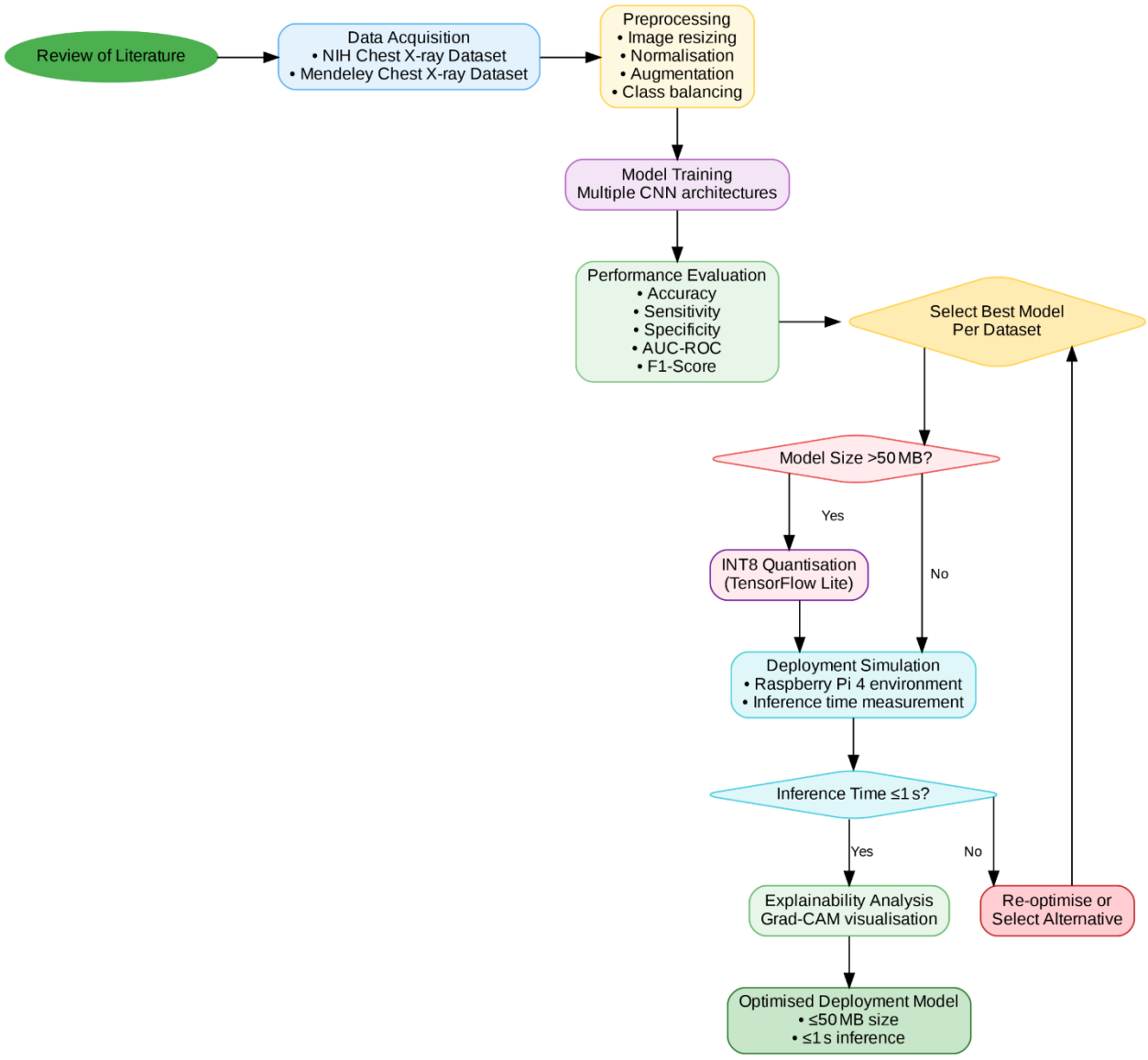


Figure 3.1: Methodology Flowchart showing the complete workflow

3.2 Datasets and Preprocessing Pipeline

3.2.1 Dataset Selection and Rationale

NIH ChestX-ray14 Dataset: A large-scale dataset comprising 112,120 frontal-view chest X-ray images from 30,805 unique patients, developed by Wang et al. (2017). Figure 3.2 demonstrates the multi-class distribution whilst Figure 3.3 highlights the severe pneumonia under-representation (1:42 ratio).

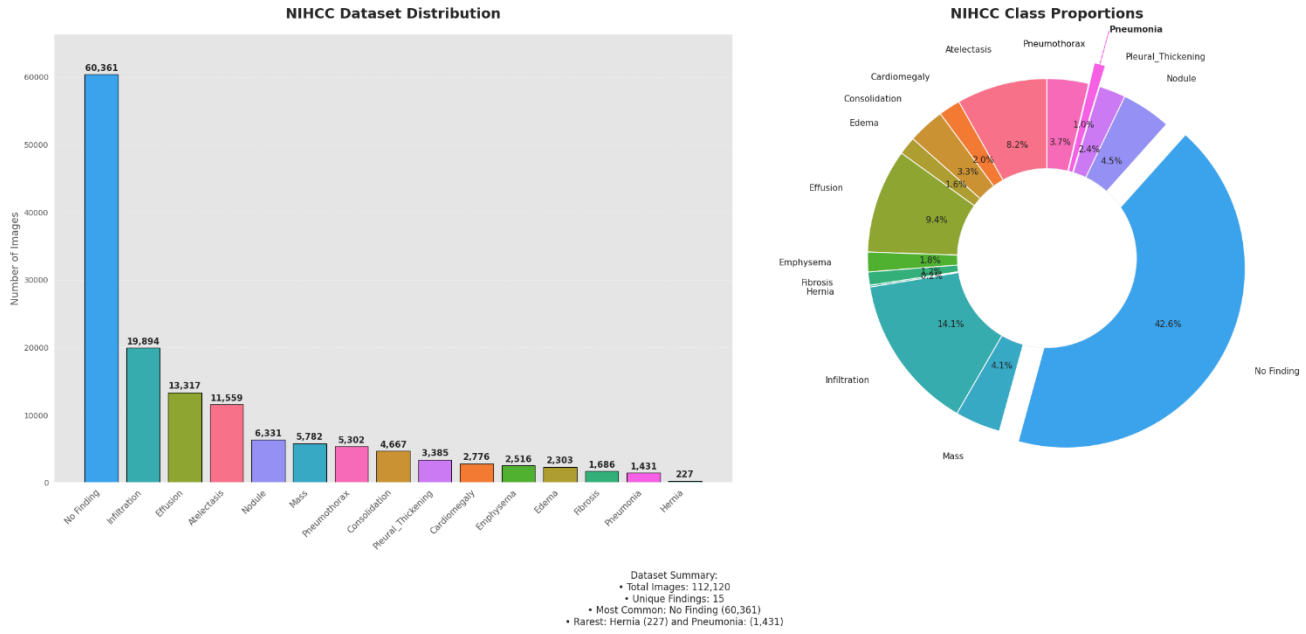


Figure 3.2: Distribution of 14 Labels in NIH ChestX-ray14 Dataset - Bar chart with annotated counts and pie chart with percentage annotations showing all 14 labels

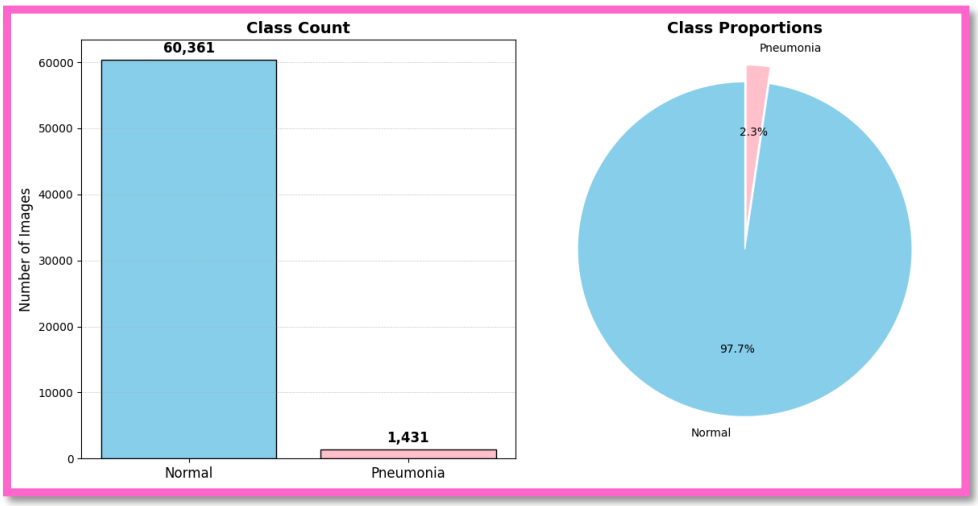


Figure 3.3: Class Distribution for Pneumonia Detection Task - Bar chart and pie chart highlighting the imbalance (60,412 Normal vs. 1,431 Pneumonia)

Mendeley Chest X-ray Dataset: A paediatric collection of 5,856 chest X-ray images curated by Kermany et al. (2018), downloaded directly from Mendeley Data repository (Version 3, DOI: 10.17632/rscbjbr9sj.3). The dataset comprises 4,672 pneumonia cases and 1,184 normal cases from Guangzhou Women and Children's Medical Centre, providing higher pneumonia prevalence (79.8%) and paediatric-specific imaging characteristics.

Figure 3.4 demonstrates the inverse class balance compared to the NIH dataset. Sample images are presented in Figures 3.5 and 3.6.

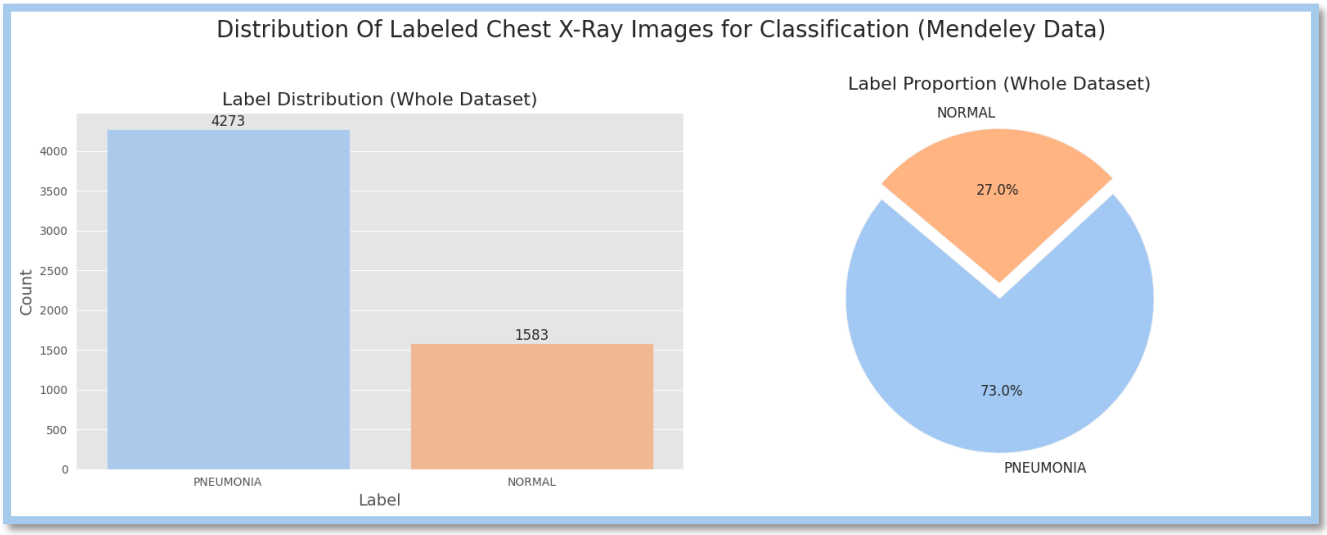


Figure 3.4: Distribution of Labeled Chest X-Ray Images (Mendeley Data) - Bar chart with counts and pie chart with percentages for Normal and Pneumonia classes

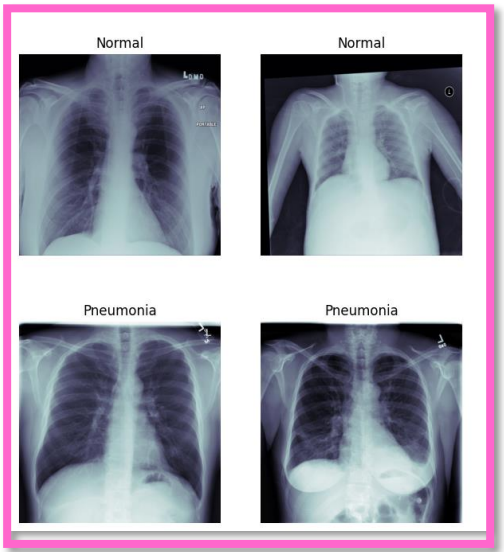


Figure 3.6: Sample X-Ray Images from NIH ChestX-ray14 Dataset

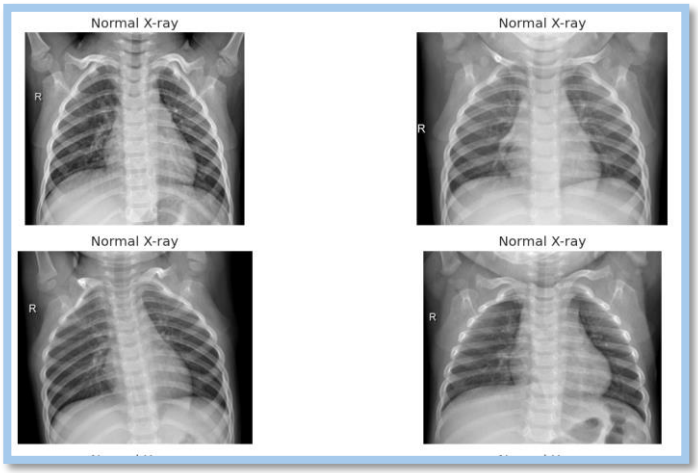


Figure 3.5: Sample X-Ray Images from Mendeley Dataset

Table 1: Dataset Characteristics and Distribution

Dataset	Total Images	Normal Cases	Pneumonia Cases	Class Ratio (N:P)	Patient Demographics
NIH ChestX-ray14	112,120	60,412	1,431	42.2:1	Adults (mixed ages)
Mendeley	5,856	1,184	4,672	1:3.9	Paediatric (1-5 years)

3.2.2 Data Splitting and Class Balancing Strategies

NIH Dataset Strategy: Cluster-based undersampling reduced severe class imbalance from 42:1 to 3:1 ratio, preserving imbalanced characteristics beneficial for medical imaging generalisation. The 80/20% split yielded: Training - 4,256 samples (3,265 normal, 991 pneumonia); Validation - 1,064 samples (804 normal, 260 pneumonia); Test - 1,784 samples (1,272 normal, 512 pneumonia).

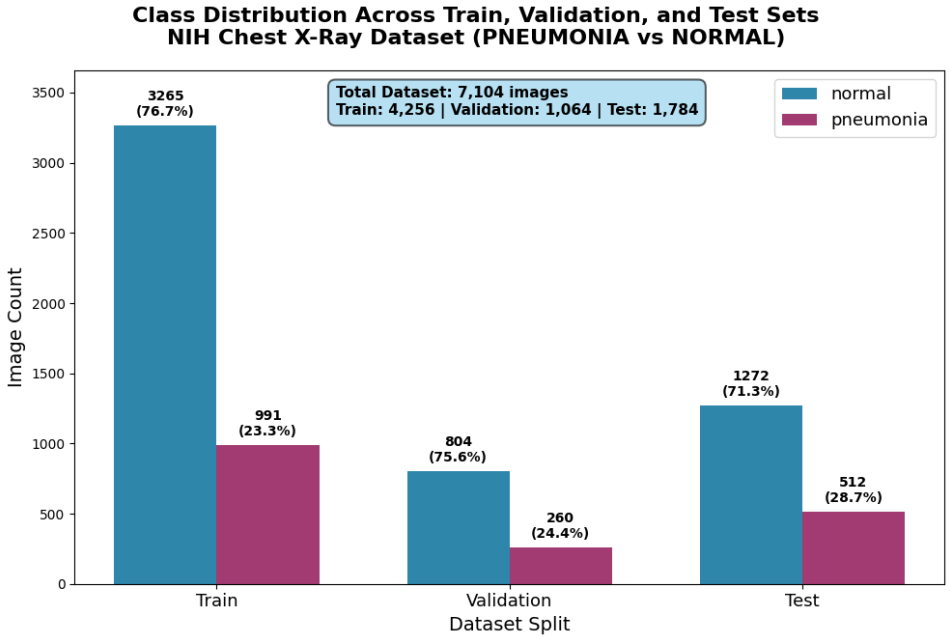


Figure 3.7: Class Distribution Across Train, Validation, and Test Sets - NIH Dataset

Mendeley Dataset Strategy: Utilising pre-existing train/test structure, 10% internal validation split yielded: Training - 4,709 samples (1,224 normal, 3,485 pneumonia); Validation - 523 samples (125 normal, 398 pneumonia); Test - 624 samples (234 normal, 390 pneumonia).

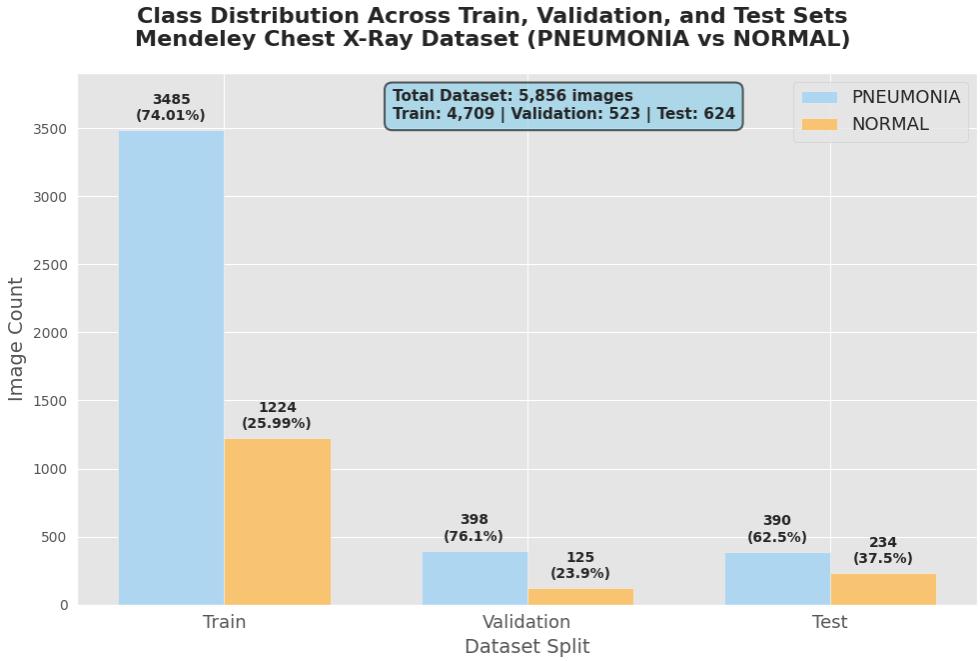


Figure 3.8: Class Distribution Across Train, Validation, and Test Sets - Mendeley Dataset

3.3 Model Architecture Design and Implementation

3.3.1 Individual Model Architectures

A comprehensive suite of ten architectures was implemented, ranging from lightweight mobile-optimised networks to state-of-the-art complex architectures.

Table 2: Model Architecture Specification

Model	Base Architecture	Classification Head Design	Total Parameters	Trainable Parameters
MobileNetV2	MobileNetV2 (ImageNet pre-trained)	GAP → Dense(256, ReLU) → Dropout(0.5) → Dense(1, sigmoid)	2,586,177	328,193
DenseNet-121	DenseNet121 (ImageNet pre-trained)	GAP → Dense(256, ReLU) → Dropout(0.5) → Dense(1, sigmoid)	7,300,161	262,657
VGG16	VGG16 (ImageNet pre-trained)	GAP → Dense(256, ReLU) → Dropout(0.5) → Dense(1, sigmoid)	14,846,273	2,491,393
ResNet50	ResNet50 (ImageNet pre-trained)	GAP → Dense(256, ReLU) → Dropout(0.5) → Dense(1, sigmoid)	49,278,337	25,690,625
InceptionV3	InceptionV3 (ImageNet pre-trained)	AdaptiveAvgPool2d → Dense(128, ReLU) → Dropout(0.2) → Dense(128, 2)	25,374,794	262,530
SE-MobileNetV2	MobileNetV2 (ImageNet pre-trained)	GAP → Dense(256, ReLU) → Dropout(0.5) → Dense(2, softmax)	2,786,114	533,250
EfficientNet B4	EfficientNet B4 (ImageNet pre-trained)	GAP → Dense(256, ReLU) → Dropout(0.5) → Dense(1, sigmoid)	18,133,088	18,007,881
SE-EfficientNetB3	EfficientNetB3 (ImageNet pre-trained)	GAP → BatchNorm → Dropout(0.3) → Dense(256, ReLU, L2) → BatchNorm → Dropout(0.4) → Dense(2, softmax)	11,479,601	4,065,634
PneumoLiteCovNet-25	Custom base architecture (PneumoLiteCovNet-25)	GAP → Dense(256, ReLU) → Dropout(0.5) → Dense(2, softmax)	2,491,234	2,465,506
Nuclear Ensemble	ResNet50, DenseNet121, InceptionV3, Xception, MobileNetV2	GAP → Dense(256, ReLU, l_2=0.01) → Dropout(0.5) → Dense(2, softmax)	77,714,002	26,875,658

3.3.2 Novel Architecture Contributions

Four novel architectural contributions enhance pneumonia detection whilst maintaining computational efficiency:

SE-MobileNetV2: Integrates Squeeze-and-Excitation blocks [30] into MobileNetV2 [31]. The novel contribution involves adaptive integration methodology where SE blocks (ratio=16) are applied to layers with sufficient channel depth, maintaining MobileNetV2's efficiency whilst enhancing feature recalibration for medical imaging.

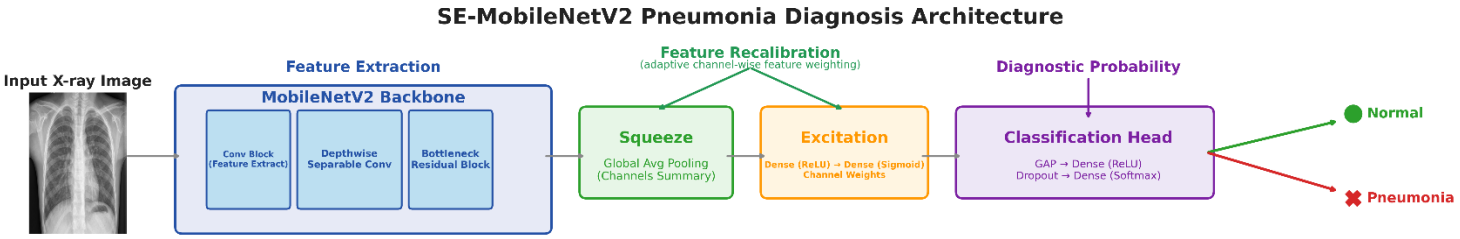


Figure 3.9: SE-MobileNetV2 Architecture

SE-EfficientNetB3: Enhances EfficientNetB3 [32] with improved SE mechanisms [30]. The contribution includes batch normalisation integration within SE blocks, L2 regularisation in SE layers, and strategic dropout implementation. The classification head employs dual batch normalisation with progressive dropout for enhanced medical imaging performance.

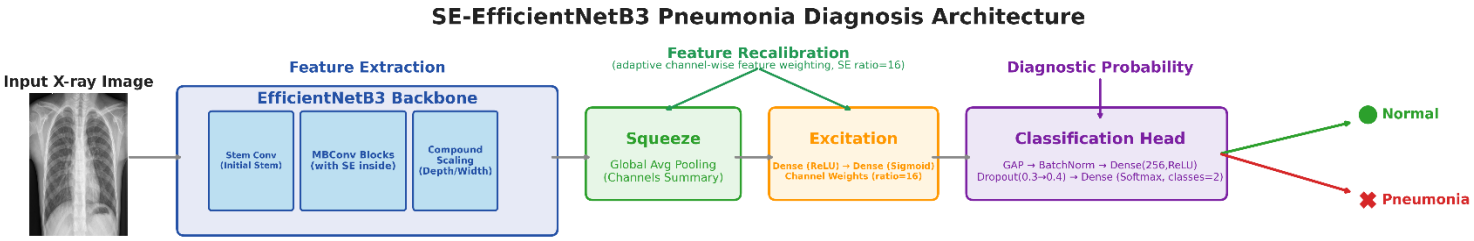


Figure 3.10: SE-EfficientNetB3 Architecture

PneumoLiteCovNet-25: A fully custom architecture combining inverted residual blocks [31], integrated SE modules [30], and novel multi-scale feature extraction. Key contributions include: multi-scale feature extraction blocks capturing 1×1, 3×3, and 5×5 features through parallel convolution branches; strategic SE placement within inverted residuals; optimised expansion factors for medical imaging yielding 2.49M parameters for edge deployment.

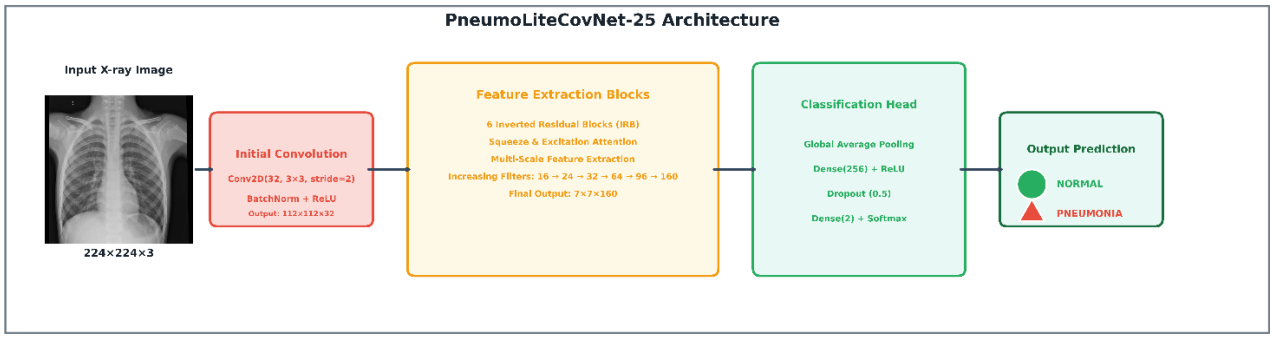


Figure 3.11: PneumoLiteCovNet-25 Architecture

Nuclear Ensemble: Meta-learning ensemble combining five base models through Optuna-optimised weighted voting [35]. Key contributions include: dual-stage prediction framework integrating automated hyperparameter optimisation for optimal model weighting with meta-learner blending; progressive meta-learner architecture with dropout regularisation; focal loss implementation for class imbalance handling; Monte Carlo dropout for uncertainty quantification during inference.

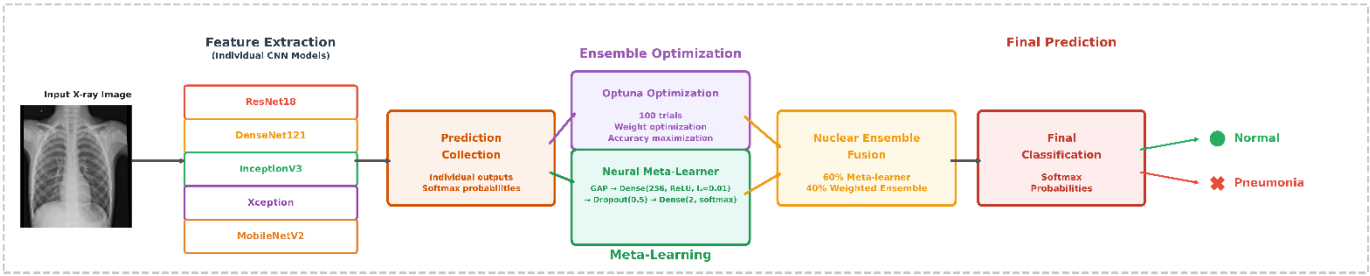


Figure 3.12: Nuclear Ensemble Architecture

3.3.3 Quantisation and Optimisation Methodology

Two distinct quantisation approaches were implemented to optimise models for edge deployment:

Dynamic Range Quantisation (INT8 Hybrid): Applied using *tf.lite.Optimize.DEFAULT*, this technique quantises model weights to INT8 precision whilst maintaining FP32 activations, achieving substantial model size reduction with minimal accuracy degradation.

Post-Training Parameter Quantisation (PT-PQ): Following Thakur et al. [13] current quantisation research methodology, this comprehensive approach quantises both weights and activations to INT8. The quantisation process maps floating-point tensors through:

$$\mathrm{A_q} = \mathrm{round}\left(\frac{\mathrm{A}}{\mathrm{scale}} + \mathrm{zero_{offset}}\right)$$

where $\boldsymbol{A_q}$ represents the quantised value of real value $\boldsymbol{A}$, scale defines the quantisation scale parameter, and $\mathbf{zero}_{\boldsymbol{offset}}$ maps real zero to integer representation. The scale parameter for bit-width $\boldsymbol{b}$ is:

$$scale = \frac{range(A)}{2^b - 1}$$

Representative calibration datasets (~520 samples) established optimal activation ranges using *tf.lite.OpsSet.TFLITE_BUILTINS_INT8* whilst preserving FP32 input/output compatibility.

3.4 Performance Evaluation Framework

3.4.1 Evaluation Metrics

A comprehensive suite of classification and discrimination metrics was employed to assess model performance across multiple dimensions:

Accuracy (A): Proportion of correct predictions over total predictions: $A = \frac{TP + TN}{TP + TN + FP + FN}$

Weighted Average Precision (P_w): Class-frequency weighted precision accounting for dataset imbalance: $P_w = \frac{\sum_{i=1}^{n} w_1 . P_i}{\sum_{i=1}^{n} w_1}$ where $\boldsymbol{w_i}$ represents class frequency weights and $\boldsymbol{P_i}$ is class-specific precision.

Sensitivity/Recall (R): Model ability to identify pneumonia cases: $R = \frac{TP}{TP + FN}$

Specificity (S): Capability to correctly identify normal cases: $S = \frac{TN}{TN + FP}$

Weighted Average F₁-Score (F1_w): Harmonic mean weighted by class frequency: $F1_w = \frac{\sum_{i=1}^{n} w_1 . F1_i}{\sum_{i=1}^{n} w_1}$

Area Under ROC Curve (AUROC): Discrimination capability computed via probability outputs, where perfect discrimination yields AUROC = 1.0 and random performance yields AUROC = 0.5.

3.4.2 Explainability Integration

Gradient-weighted Class Activation Mapping (Grad-CAM) [33] enables clinical interpretability through visual explanation of diagnostic decisions. Implementation employs systematic target layer identification and TensorFlow's automatic differentiation for precise pneumonia localisation. Resolution optimisation strategies maintain explanation fidelity whilst reducing processing overhead, enabling real-time interpretability generation essential for clinical decision support and physician trust in resource-constrained environments.

4. Implementation

4.1 Software Architecture and Development Environment

Python 3.8+ with TensorFlow 2.x provided the core framework, supplemented by PyTorch for specific architectural components. Training utilised Google Colab Pro GPU acceleration for complex models, whilst CPU environments enabled edge deployment simulation. Development incorporated Scikit-learn for metrics, NumPy and Pandas for data manipulation.

4.2 Model Training Implementation

Transfer learning employed ImageNet pre-trained weights with strategic fine-tuning approaches. Several base architectures were adapted from existing GitHub implementations and modified for pneumonia detection: VGG16 [36], InceptionV3 [37], MobileNetV2 [38], and ResNet50 [39]. AI-assisted development provided foundational architectural guidance which was subsequently adapted for specific pneumonia detection requirements, including custom hyperparameter adjustments and variable modifications to ensure consistency across model implementations (AI assistance details in Appendix). Novel architectural contributions (SE-MobileNetV2, SE-EfficientNetB3, PneumoLiteCovNet-25, Nuclear Ensemble) were developed through iterative refinement of these base structures. The code used in the implementation and training of the models is available on GitHub[1]

4.3 Implementation Challenges

Implementation presented several technical complexities requiring systematic solutions. SE-Block integration utilised a reduction ratio of 16 with L2 regularisation and dropout for improved generalisation (Figure 4.1). Nuclear Ensemble coordination managed different input size requirements, with InceptionV3 and Xception using 299×299 pixel inputs whilst other models used 224×224 dimensions (Figure 4.2). PneumoLiteCovNet-25 implemented multi-scale feature extraction through parallel convolution branches using 1×1, 3×3, and 5×5 kernel sizes with channel concatenation (Figure 4.3).

[1] GitHub Repository: https://github.com/EvansKonadu/AI-Powered-Pneumonia-Detection-in-Low-Resource-Settings

```python
# Squeeze and Excitation block
def squeeze_excite_block(input_tensor, ratio=16):
    init = input_tensor
    channel_axis = -1
    filters = init.shape[channel_axis]
    se_shape = (1, 1, filters)

    se = GlobalAveragePooling2D()(init)
    se = Reshape(se_shape)(se)
    se = Dense(filters // ratio, activation='relu',
kernel_initializer='he_normal',
               use_bias=False,
kernel_regularizer=tf.keras.regularizers.l2(1e-4))(se)
    se = Dropout(0.2)(se)  # Added dropout for regularization
    se = Dense(filters, activation='sigmoid',
kernel_initializer='he_normal', use_bias=False)(se)

    x = multiply([init, se])
    return x
```

Figure 4.1: SE-Block integration code

```python
# Model creation with Monte Carlo Dropout
def create_densenet121_model(input_shape=(224, 224, 3),
classes=2):
    base_model = DenseNet121(input_shape=input_shape,
include_top=False, weights='imagenet')

    for layer in base_model.layers[:-30]:
        layer.trainable = False

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation='relu', kernel_regularizer=l2(0.01))(x)
    x = Dropout(0.5)(x)  # Monte Carlo Dropout
    x = Dense(classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=x)
    return model

def create_inceptionv3_model(input_shape=(299, 299, 3), classes=2):
    base_model = InceptionV3(input_shape=input_shape,
include_top=False, weights='imagenet')

    for layer in base_model.layers[:-30]:
        layer.trainable = False

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation='relu', kernel_regularizer=l2(0.01))(x)
    x = Dropout(0.5)(x)  # Monte Carlo Dropout
    x = Dense(classes, activation='softmax')(x)

    model = Model(inputs=base_model.input, outputs=x)
    return model

def create_xception_model(input_shape=(299, 299, 3),
classes=2):
    base_model = Xception(input_shape=input_shape, include_top=False,
weights='imagenet')
```

Figure 4 2: Nuclear Ensemble dual input handling code

```python
# Multi-scale feature extraction block
def multi_scale_block(inputs, filters):
    # Branch 1: 1x1 convolution
    branch1 = Conv2D(filters, kernel_size=1,
padding='same')(inputs)
    branch1 = BatchNormalization()(branch1)
    branch1 = Activation('relu')(branch1)

    # Branch 2: 3x3 convolution
    branch2 = Conv2D(filters, kernel_size=3,
padding='same')(inputs)
    branch2 = BatchNormalization()(branch2)
    branch2 = Activation('relu')(branch2)

    # Branch 3: 5x5 convolution (implemented as two 3x3 convs)
    branch3 = Conv2D(filters, kernel_size=3, padding='same')(inputs)
    branch3 = BatchNormalization()(branch3)
    branch3 = Activation('relu')(branch3)
    branch3 = Conv2D(filters, kernel_size=3, padding='same')(branch3)
    branch3 = BatchNormalization()(branch3)
    branch3 = Activation('relu')(branch3)

    # Concatenate branches
    x = Concatenate(axis=-1)([branch1, branch2, branch3])

    # 1x1 convolution to reduce channels
    x = Conv2D(filters, kernel_size=1, padding='same')(x)
    x = BatchNormalization()(x)
    x = Activation('relu')(x)

    return x
```

Figure 4.3: Multi-scale feature extraction code

4.4 Quantisation and Edge Deployment

4.4.1 TensorFlow Lite Implementation

Model quantisation utilised systematic approaches with representative datasets for activation range calibration. Conversion incorporated error handling for unsupported operations and fallback mechanisms for complex components.

4.4.2 Edge Deployment Simulation

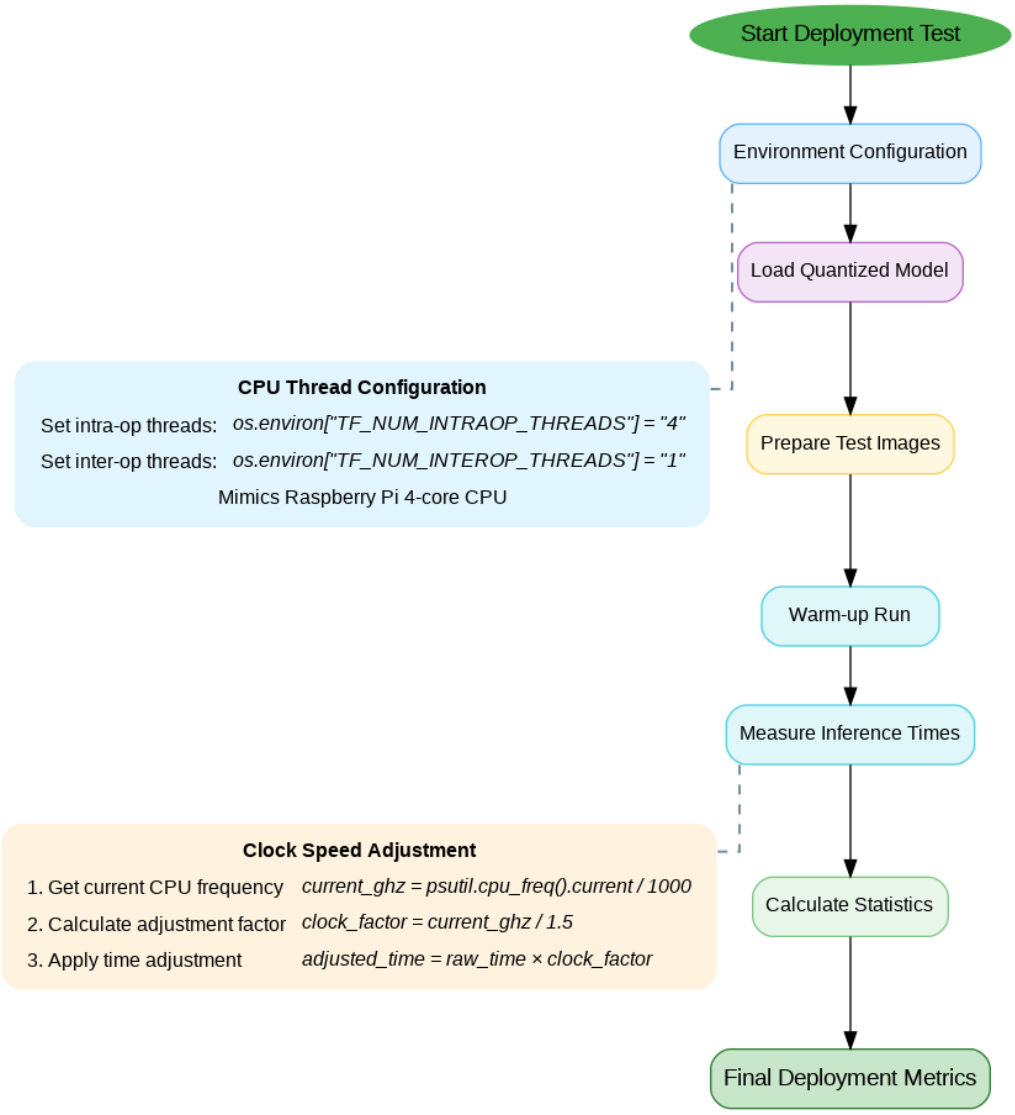


Figure 4.4: Edge Deployment Workflow

CPU environments were configured to emulate Raspberry Pi 4B specifications (1.5GHz, 4-core constraints), as illustrated in Figure 4.4, enabling realistic inference assessment under edge deployment conditions.

Table 3: Computational Platform Specifications for Edge Deployment Evaluation

Hardware Specification	Development Platform (Colab CPU)	Target Edge Platform (Raspberry Pi 4B)
CPU Architecture	Intel x86_64 @ 2.2 GHz	Broadcom BCM2711 SoC (Cortex-A72 Quad-Core ARM v8) @ 1.5 GHz
Processing Cores	1 physical core, 2 logical threads	4 physical cores, 4 logical threads
Memory Configuration	12.67 GB LPDDR4, x86_64 architecture	4 GB LPDDR4-3200, ARM 64-bit architecture
Primary Use	Model development and baseline inference	Target edge deployment simulation

4.5 Explainability Implementation

Grad-CAM implementation utilised TensorFlow's GradientTape with systematic target layer identification across architectures. Computational efficiency was achieved through resolution optimisation, computing gradients at reduced dimensions before upsampling for clinical display [34]. OpenCV (v4.8.0) [40] enabled heatmap generation with semi-transparent overlay (α=0.4) and class-specific colour coding. Custom methods ensured consistent behaviour across architectures with error handling for unsupported components, enabling seamless integration with the inference pipeline.

4.6 Data Management and Ethics

All datasets are publicly available with appropriate institutional ethical approval. No patient identifiers were accessed, and data handling adhered to general protection principles and medical image analysis best practices.

5. Experiments, Results and Discussion

5.1 Data Preprocessing and Augmentation Implementation

All images underwent standardised preprocessing ensuring consistency across architectures. Resolution normalisation targeted 224×224 pixels for most models, with 299×299 for InceptionV3 and Xception. Preprocessing employed pixel normalisation (x/255.0) with ImageNet standardisation for transfer learning compatibility.

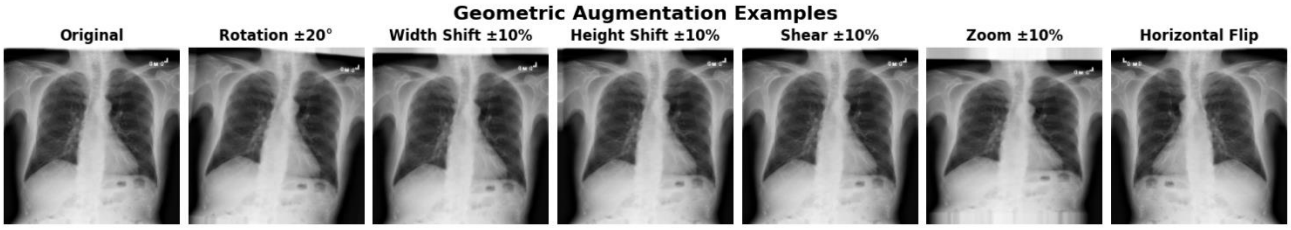


Figure 5.1: Geometric Data Augmentation Examples - Demonstrating rotation, zoom, shift, and shear transformations applied whilst preserving anatomical orientation

Geometric augmentation enhanced generalisation through medical imaging-appropriate transformations: rotation (±20°), zoom (±10%), spatial shifts (±10%), shear (±10%), and horizontal flipping. Vertical flipping was excluded to preserve anatomical orientation. TensorFlow's ImageDataGenerator enabled systematic augmentation with balanced probability distributions, constraining parameters to maintain realistic clinical imaging variations.

5.2 Training Configuration and Performance Analysis

Training configurations employed batch size 32 with class weighting and focal loss for imbalance mitigation. Early stopping (patience 5-7 epochs) with ModelCheckpoint callbacks optimised convergence whilst preventing overfitting. The best epoch for each model was determined by selecting the epoch with the lowest validation loss, ensuring optimal generalisation performance whilst mitigating overfitting risk. The instances where validation loss was lower than training loss in SE-EfficientNetB3, SE-MobileNetV2 and PneumoLiteCovNet25 (Table 4) resulted from effective regularisation through dropout and L2 penalties being active during training, combined with focal loss behaviour on the imbalanced dataset.

Table 4: NIH ChestX-ray14 Dataset Training and Evaluation Performance Summary (Metrics at Best Epoch)

(Bold = Best | Underline = 2nd Best)

Model	Total Epochs	Best Epoch	Train Loss	Val Loss	Val Accuracy	Test Loss	Test Accuracy	Model Size (MB)	Device	Train Time (min)
MobileNetV2	20	15	0.5700	0.5654	0.7330	0.5559	0.7405	**11.01**	CPU	38.75
DenseNet-121	30	2	0.5796	0.5846	0.6971	0.5613	0.7169	31.12	GPU	90.50
VGG16	30	11	0.5695	0.5004	0.7566	0.5102	<u>0.7556</u>	75.74	GPU	204.53
ResNet50	20	8	0.6432	0.5154	0.7686	0.5666	0.7253	494.88	GPU	57.45
InceptionV3	30	9	0.5998	0.6072	0.7397	0.5308	0.7371	97.13	GPU	310.16
SE-MobileNetV2	25	25	0.8982	0.4833	0.8125	0.5434	0.7438	<u>15.27</u>	CPU	117.58
EfficientNet B4	15	1	0.0281	0.4749	0.7707	0.6920	**0.8610**	208.32	GPU	204.25
SE-EfficientNetB3	30	11	0.3196	0.0920	0.9223	0.1442	0.7377	76.14	GPU	173.71
PneumoLiteCovNet-25	25	18	1.1882	0.6552	0.6461	0.6535	0.6586	29.23	CPU	141.64

Table 5: Mendeley Dataset Training and Evaluation Performance Summary (Metrics at Best Epoch)

(Bold = Best | Underline = 2nd Best)

Model	Total Epochs	Best Epoch	Train Loss	Val Loss	Val Accuracy	Test Loss	Test Accuracy	Model Size (MB)	Device	Train Time (min)
VGG16	30	8	0.0654	0.0907	0.9618	0.3150	0.8926	75.74	CPU	913.90
DenseNet121	30	15	0.1416	0.0955	0.9636	0.2012	0.9151	31.12	GPU	125.69
MobileNetV2	20	15	0.1703	0.1106	0.9531	0.1865	0.9151	**12.89**	CPU	71.58
InceptionV3	30	29	0.2054	0.1845	0.9389	0.3202	0.8750	96.80	GPU	70.99
ResNet50	20	8	0.1491	0.1387	0.9426	0.2981	0.8686	494.88	GPU	56.47
EfficientNetB4	15	2	0.0255	0.0839	0.9673	0.1413	**0.9519**	208.44	GPU	496.20
SE-MobileNetV2	20	18	0.1766	0.0236	0.9961	0.1527	<u>0.9407</u>	<u>15.27</u>	CPU	150.99
SE-EfficientNetB3	30	21	0.0757	0.0532	0.9981	0.0748	0.9247	76.14	CPU	639.20
PneumoLiteCovNet-25	25	15	0.4679	0.5611	0.8730	0.4662	0.8285	29.23	CPU	303.60

Training dynamics revealed dataset-specific patterns. Mendeley achieved superior convergence with lower losses and higher accuracies, reflecting paediatric focus and higher pneumonia prevalence. NIH presented challenges from severe class imbalance, resulting in extended training and conservative metrics. EfficientNetB4 achieved 97.3% validation accuracy on Mendeley with early convergence, maintaining 77.1% on NIH despite imbalance challenges.

5.3 Ensemble Architecture Analysis

Table 6: Nuclear Ensemble Component Training Performance Summary (Metrics at Best Epoch)

(Bold = Best | Underline = 2nd Best)

Dataset	Model	Total Epochs	Best Epoch	Train Loss	Val Loss	Val Accuracy	Test Loss	Test Accuracy	Model Size (MB)
***Mendeley Data* [12]**	ResNet50	15	15	0.0098	0.0060	0.9770	0.1854	0.9375	92.61
	DenseNet121	15	14	0.0127	0.0060	0.9809	0.2614	0.9087	<u>29.29</u>
	InceptionV3	15	15	0.0123	0.0069	0.9627	0.2961	0.9359	86.22
	Xception	15	14	0.0073	0.0046	0.9837	0.1839	**0.9696**	82.06
	MobileNetV2	15	12	0.0146	0.0131	0.9330	0.2208	0.9183	**10.42**
	Nuclear Ensemble						0.1435	<u>0.9679</u>	300.74
***NIH ChestX-ray14* [6]**	ResNet50	15	13	0.0260	0.0317	0.8015	0.4751	**0.8016**	91.98
	DenseNet121	15	12	0.0277	0.0324	0.7912	0.5156	0.7820	<u>27.85</u>
	InceptionV3	15	6	0.0283	0.0320	0.7827	0.5275	0.7691	85.17
	Xception	15	5	0.0260	0.0313	0.8062	0.4946	<u>0.7954</u>	81.58
	MobileNetV2	15	6	0.0293	0.0373	0.7357	0.5639	0.7472	**9.87**
	Nuclear Ensemble						0.4642	0.7870	505.89

Table 7: Nuclear Ensemble Component Performance Comparison - Showing individual and combined performance across architectures

Dataset	Model	Accuracy (%)	Precision (w.AVG) (%)	Sensitivity (%)	Specificity (%)	F1-score (w.AVG) (%)	AUROC
***Mendeley Data* [12]**	ResNet50	93.75	94.17	91.79	97.01	94.83	0.9885
	DenseNet121	90.87	91.80	87.69	96.15	92.31	0.9855
	InceptionV3	93.59	93.58	95.13	91.03	94.88	0.9807
	Xception	**96.96**	96.98	96.92	97.01	**97.55**	**0.9933**
	MobileNetV2	91.83	91.87	92.82	90.17	93.42	0.9740
	Nuclear Ensemble	<u>96.79</u>	98.18	96.67	97.01	97.42	<u>0.9915</u>
***NIH ChestX-ray14* [6]**	ResNet50	**80.16**	79.27	51.56	91.67	**79.08**	0.8353
	DenseNet121	78.20	77.05	42.19	92.69	76.31	0.8364
	InceptionV3	76.91	75.79	33.53	94.34	73.91	0.8242
	Xception	<u>79.54</u>	78.61	42.27	92.53	<u>78.09</u>	<u>0.8436</u>
	MobileNetV2	74.72	76.93	68.36	77.28	75.45	0.7930
	Nuclear Ensemble	78.70	78.10	38.67	94.81	76.24	**0.8529**

Nuclear Ensemble combined five diverse architectures (ResNet50, DenseNet121, InceptionV3, Xception, MobileNetV2) through performance-weighted voting. Mendeley training achieved superior base model performance (>93% validation accuracy), whilst NIH demonstrated conservative performance due to class imbalance. The ensemble achieved 96.79% accuracy with 0.9915 AUROC on Mendeley through effective meta-learner integration. NIH performance (78.70% accuracy) highlighted ensemble coordination challenges for imbalanced datasets, where conservative base predictions influenced overall behaviour toward high specificity

at reduced sensitivity. This pattern suggests ensemble utility for clinical scenarios requiring high-confidence positive diagnoses.

5.4 Comparative Performance Analysis Across Datasets

Performance evaluation across the paediatric-focused Mendeley dataset and the adult-population NIH ChestX-ray14 dataset to assess architecture-specific deployment suitability.

Table 8: Comparative Performance Metrics Across Datasets (Pre-Quantisation) - Showing performance variations between paediatric and adult populations

(Bold = Best | Underline = 2nd Best)

Dataset	Model	Accuracy (%)	Precision (w.AVG) (%)	Sensitivity (%)	Specificity (%)	F1-Score (w.AVG) (%)	AUROC
Mendeley Data (Paediatric) [12]	Nuclear Ensemble	<u>96.79</u>	98.18	96.67	97.01	<u>97.42</u>	<u>0.9915</u>
	Xception	**96.96**	96.98	96.92	97.01	**97.55**	**0.9933**
	EfficientNetB4	95.19	95.23	97.95	90.60	95.16	0.9887
	SE-MobileNetV2	94.07	94.06	95.90	91.03	94.06	0.9851
	MobileNetV2	91.51	91.69	96.92	82.48	91.38	0.9818
	SE-EfficientNetB3	92.47	92.52	96.41	85.90	92.40	0.9818
	DenseNet121	91.51	91.73	<u>97.18</u>	82.05	91.37	0.9789
	VGG16	89.26	90.71	**99.74**	71.79	88.81	0.9815
	ResNet50	87.50	89.28	82.82	95.30	87.68	0.9705
	InceptionV3	87.50	88.06	96.41	72.65	87.13	0.9507
	PneumonLiteCovNet-25	82.85	83.37	94.10	64.10	82.19	0.9137
NIH ChestX-ray14 (Adults) [6]	Nuclear Ensemble	78.70	78.10	38.67	94.84	76.24	<u>0.8528</u>
	Xception	<u>79.54</u>	78.61	47.27	92.53	<u>78.09</u>	0.8364
	EfficientNetB4	**86.10**	86.10	**78.91**	88.99	**86.22**	**0.8986**
	SE-MobileNetV2	74.38	75.80	63.67	78.69	74.92	0.7854
	MobileNetV2	74.05	75.70	<u>64.26</u>	77.99	74.66	0.7833
	SE-EfficientNetB3	73.77	71.42	33.59	89.94	71.35	0.7399
	DenseNet121	71.69	73.05	58.20	77.12	72.24	0.7447
	VGG16	75.56	76.07	60.74	81.53	75.79	0.7949
	ResNet50	72.53	72.93	54.49	79.80	72.72	0.7423
	InceptionV3	73.71	72.84	48.24	83.96	73.71	0.7519
	PneumonLiteCovNet-25	65.86	70.88	63.28	66.90	67.31	0.6896

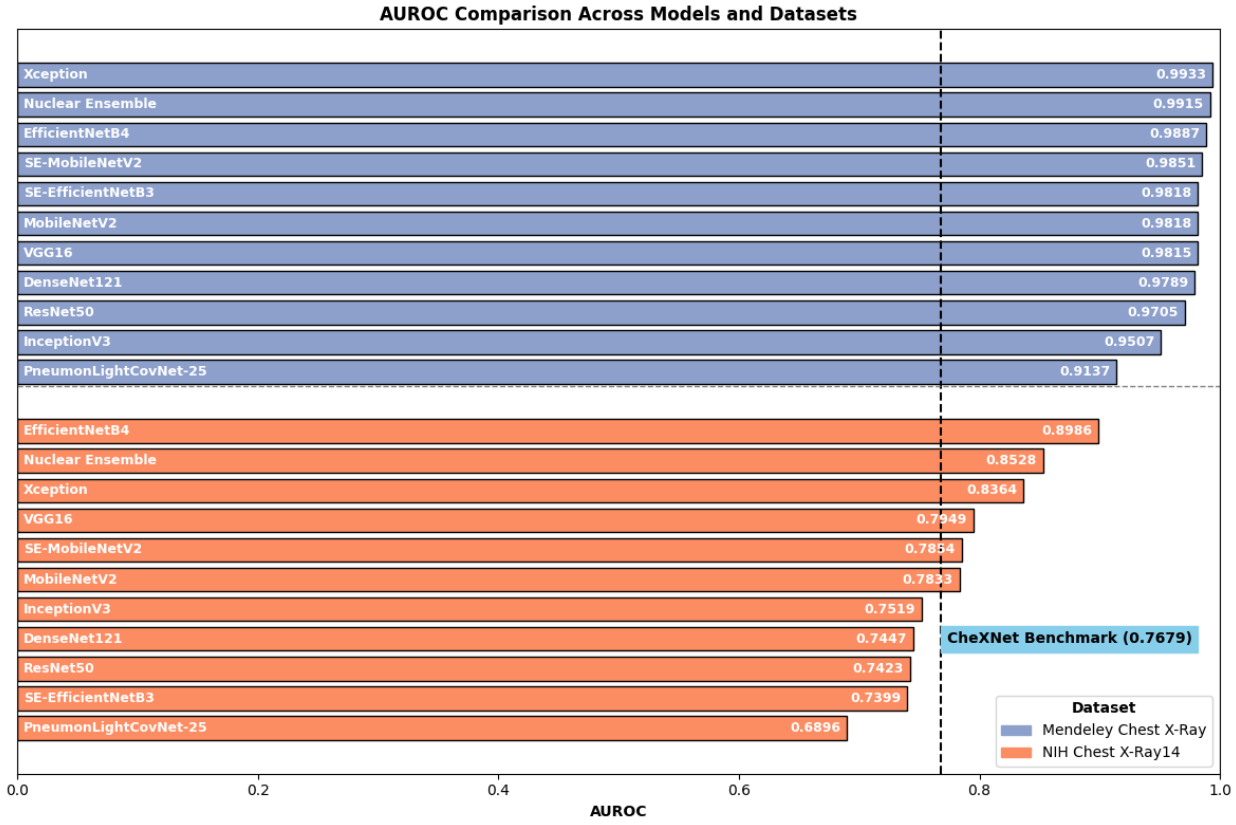


Figure 5.2: Comparative AUROC Performance Across Datasets with CheXNet Benchmark - Showing performance differential between paediatric and adult imaging

Performance analysis revealed substantial dataset-specific characteristics. Mendeley consistently enabled 10-20% accuracy improvements relative to NIH, reflecting paediatric imaging characteristics and higher pneumonia prevalence. Notably, EfficientNetB4 demonstrated consistent multi-dataset robustness, achieving third-best performance on Mendeley (95.19%) whilst excelling as the optimal architecture on NIH (86.10%), indicating superior generalisation capabilities across demographic groups. The performance differential indicates architecture-specific deployment advantages for distinct clinical demographics, though raises considerations for cross-demographic deployment effectiveness in diverse healthcare settings.

5.5 Model Optimisation Through Quantisation

Table 9: Original vs Quantised Model Performance Comparison - Demonstrating size reduction and performance preservation across architectures

Model	Size (MB)	Accuracy (%)	Sensitivity (%)	Specificity (%)	Precision *(w.AVG)* (%)	F1-Score *(w.AVG)* (%)	AUC
Orig. Xception	82.08	96.96	96.92	97.01	98.18	97.55	0.9933
Quant. Xception	21.61	97.12	97.69	96.15	97.69	97.69	0.9934
Orig. Ensemble	300.74	96.79	96.67	97.01	96.83	96.80	0.9915
Quant. Ensemble	77.17	95.99	97.69	93.16	95.97	96.82	0.9925
Orig. EffNetB4	208.32	86.10	78.91	88.99	86.40	86.22	0.8986
Quant. EffNetB4	18.59	80.94	72.66	84.28	65.03	68.63	0.8563

Post-Training Parameter Quantisation (PT-PQ) demonstrated architecture-dependent effectiveness. Xception achieved remarkable improvement from 96.96% to 97.12% accuracy whilst reducing from 82.08 MB to 21.61 MB (74% reduction), likely reflecting quantisation's regularisation benefits. Nuclear Ensemble maintained comparable performance (96.79% to 95.99%) with substantial 300.74 MB to 77.17 MB reduction. EfficientNetB4 required hybrid INT8 approach, achieving 86.10% to 80.94% performance with 208.32 MB to 18.59 MB compression (91% reduction). Performance degradation reflected architectural complexity and quantisation sensitivity, where EfficientNet's compound scaling approach demonstrated greater sensitivity to precision reduction compared to Xception's depthwise separable architecture.

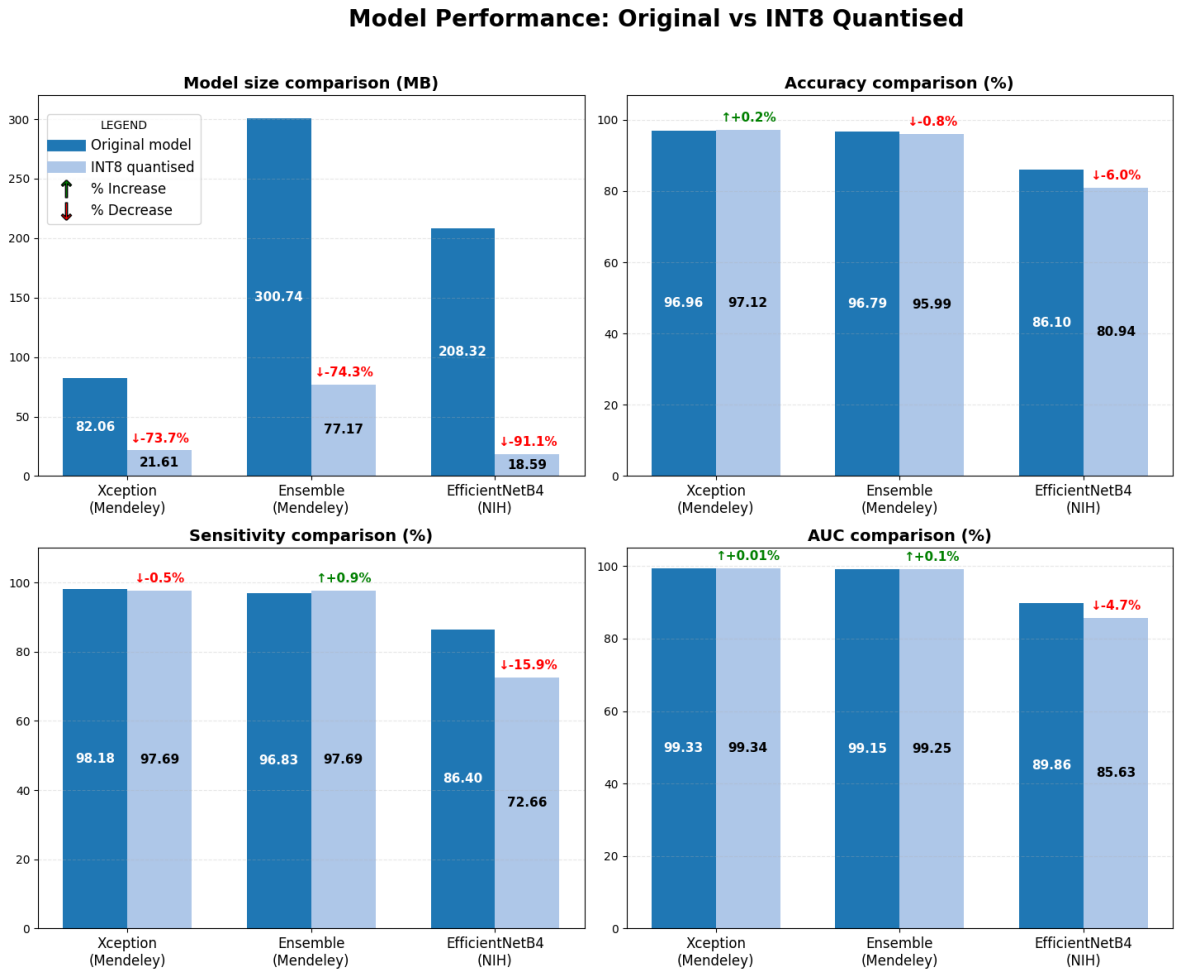


Figure 5.3: Quantisation Impact Analysis - Comparing original versus quantised model performance across key metrics

5.6 Edge Deployment Performance Evaluation

Table 10: Edge Deployment Inference Performance - Showing inference times on Raspberry Pi-simulated environments

Model	Dataset	Size (MB)		Adj. Inference Time (ms)	
		Original	Quantised	Original	Quantised
Nuclear Ensemble	Mendeley	300.74	77.17	5230.78 ± 828.11	1646.77 ± 409.74
Xception	Mendeley	82.06	21.61	1038 .79 ± 205.36	550.76 ± 119.67
EfficientNetB4	NIH	208.32	18.59	1461.07 ± 425.52	232.69 ± 93.50

Quantised models achieved substantial inference improvements enabling clinical deployment. Xception improved from 1038.79 ± 205.36 ms to 550.76 ± 119.67 ms (47% improvement), whilst EfficientNetB4 achieved 1461.07 ± 425.52 ms to 232.69 ± 93.50 ms (84% improvement). These enable practical Raspberry Pi deployment with sub-second response times. Nuclear Ensemble's original 5.2-second inference remained unsuitable despite quantisation improvements to 1.6 seconds, highlighting computational trade-offs between ensemble accuracy and edge deployment feasibility. The inference variability reflects TensorFlow Lite's dynamic optimisation under simulated edge conditions.

5.7 Explainability Integration and Performance

Gradient-weighted Class Activation Mapping (Grad-CAM) achieved substantial computational efficiency improvements whilst maintaining explanation fidelity for clinical decision support. The resolution optimisation approach reduced computational complexity proportional to spatial dimensions squared, with theoretical predictions of 3-5× speedup further enhanced by *tf.function* compilation optimisation and memory access improvements [34]. Empirical evaluation demonstrated exceptional performance: Xception achieved 5.06× speedup (2.756 ± 0.630 to 0.544 ± 0.144 seconds) with high correlation (0.960) and IoU@0.5 (0.960), whilst EfficientNetB4 achieved 14.56× speedup (5.417 ± 2.218 to 0.372 ± 0.127 seconds) with perfect fidelity metrics, enabling real-time clinical explanation generation essential for physician trust and point-of-care diagnostic support in resource-constrained environments.

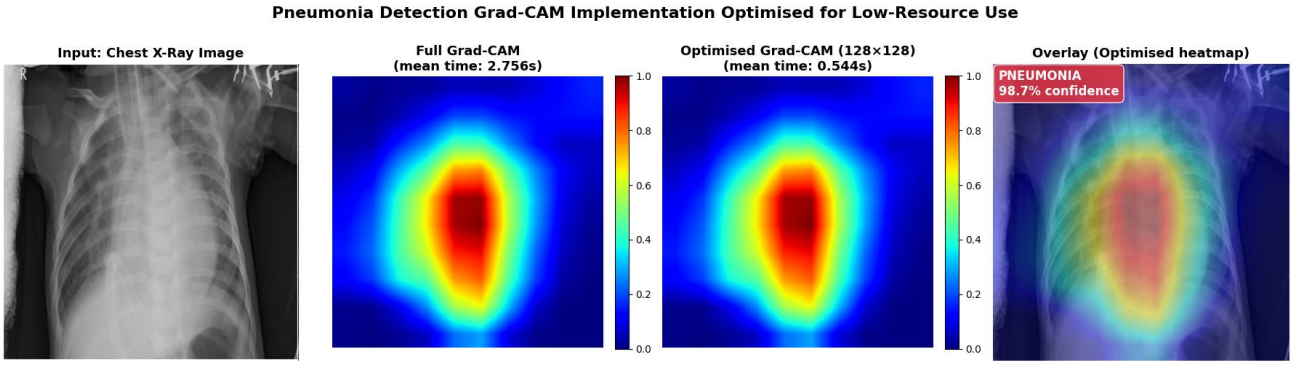


Figure 5.4: Grad-CAM Attention Visualisations - Xception (Mendeley Dataset) - Demonstrating model attention regions for pneumonia detection in paediatric chest X-rays

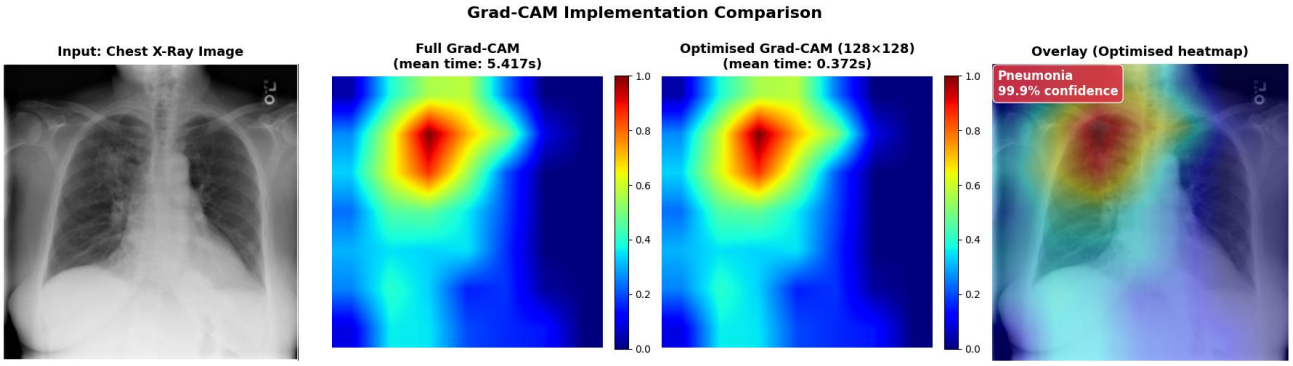


Figure 5.5: Grad-CAM Attention Visualisations - EfficientNetB4 (NIH Dataset) - Showing interpretability maps for adult chest X-ray analysis

5.8 Optimal Architecture Selection and Clinical Deployment

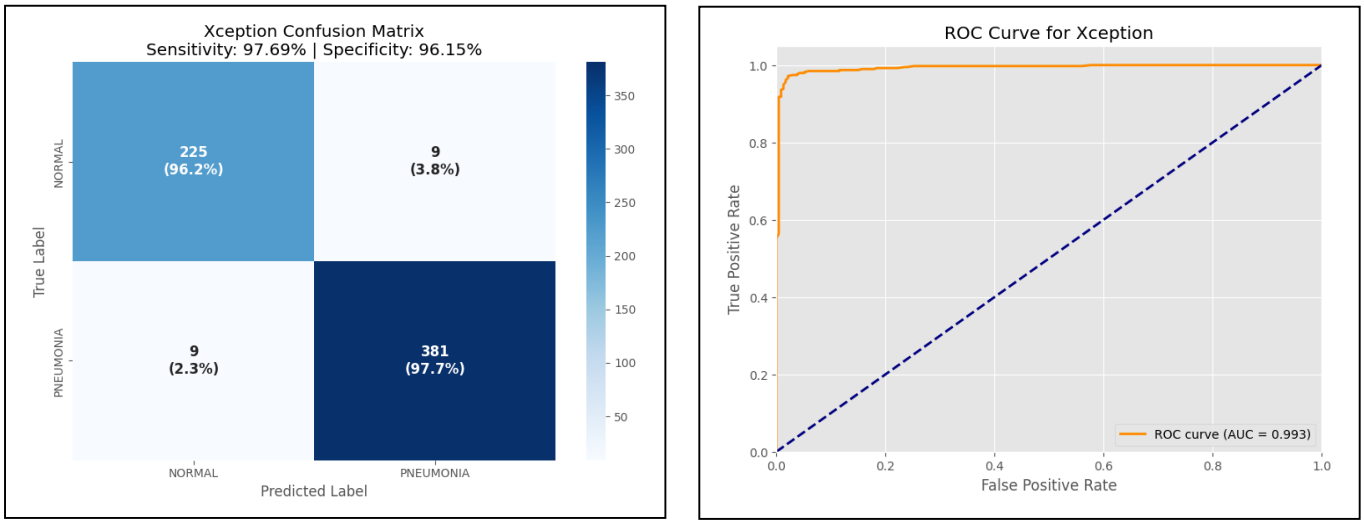


Figure 5.6: Confusion Matrix and ROC Analysis - Xception (Mendeley Dataset)

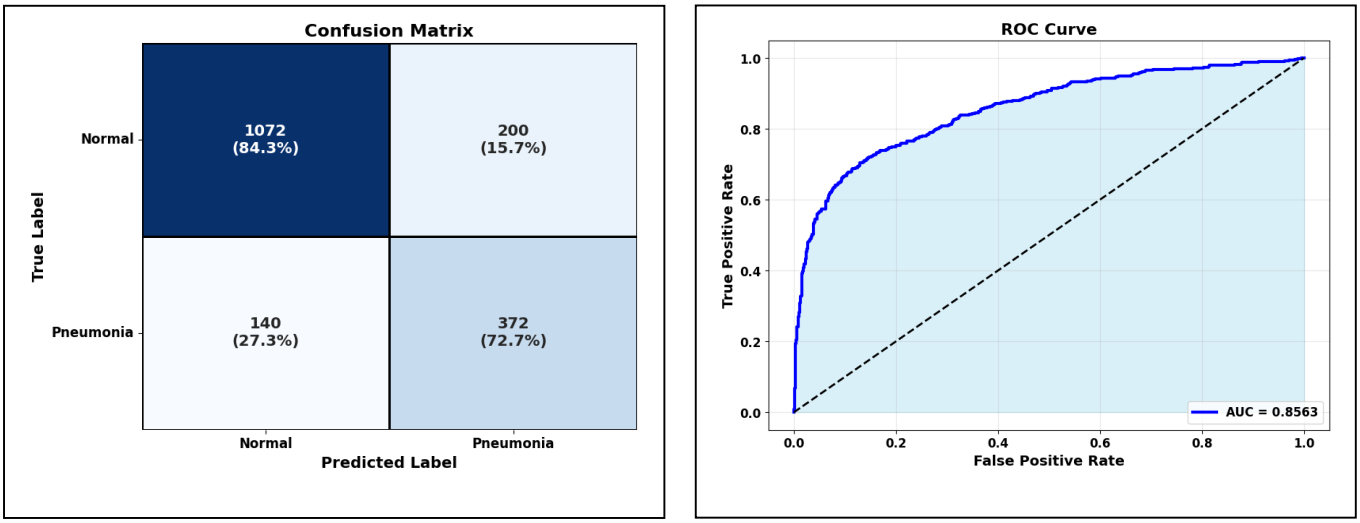


Figure 5.7: Confusion Matrix and ROC Analysis - EfficientNetB4 (NIH Dataset)

Xception (Quantised) emerged optimal for paediatric applications on the Mendeley dataset with 97.12% accuracy, 21.61 MB size, and 550.76 ms inference. Superior sensitivity (97.69%) suits childhood screening where missed diagnoses carry severe consequences. The architecture demonstrated clinical interpretability and edge compatibility, though the 299×299 input requirement creates computational overhead compared to standard 224×224 processing. EfficientNetB4 (Quantised**)** proved optimal for adult applications on the NIH dataset with 80.94% accuracy, 18.59 MB size, and 232.69 ms inference. Balanced sensitivity-specificity profile suits general screening applications. Both architectures fulfilled deployment requirements: diagnostic accuracy, computational efficiency, and clinical interpretability essential for resource-constrained healthcare settings. Comparative evaluation across datasets revealed substantial performance differentials requiring careful deployment consideration. The 15-20% accuracy variation between datasets indicates population-specific optimisation needs, whilst Nuclear Ensemble's computational demands (>1.6 seconds post-quantisation) limit point-of-care applications despite superior accuracy on balanced datasets.

6. Conclusions and Future Work

This research successfully developed and optimised deep learning models for pneumonia detection in resource-constrained settings, achieving the primary objective of balancing diagnostic accuracy with computational efficiency. Two architectures emerged as optimal solutions: Xception for paediatric applications (97.12% accuracy, 21.61 MB) and EfficientNetB4 for adult scenarios (80.94% accuracy, 18.59 MB), both demonstrating edge deployment feasibility with sub-second inference times and integrated explainability mechanisms.

Key contributions include systematic multi-dataset evaluation revealing architecture-specific deployment advantages, systematic quantisation evaluation achieving up to 91% size reduction whilst maintaining clinical utility, and comprehensive edge deployment validation on Raspberry Pi-simulated environments. Novel pipeline contributions encompass the development of SE-MobileNetV2 and SE-EfficientNetB3 architectures with adaptive squeeze-and-excitation integration, the design of PneumoLiteCovNet-25 featuring multi-scale feature extraction blocks and optimised inverted residuals, and the implementation of Nuclear Ensemble meta-learning framework combining five diverse architectures through performance-weighted voting with Monte Carlo dropout uncertainty quantification. Additionally, the optimised explainability integration pipeline enabling real-time Grad-CAM generation with 5-15× computational speedup represents a significant contribution to clinical interpretability in resource-constrained environments. The research demonstrates that clinically acceptable pneumonia detection can be achieved within severe computational constraints, supporting healthcare democratisation in low-resource settings where traditional diagnostic infrastructure is unavailable.

Current limitations encompass population-specific performance variations requiring cross-demographic validation, architectural constraints including Xception's non-standard input dimensions and ensemble computational overhead, and dataset-dependent quantisation effectiveness that suggests architecture-specific optimisation strategies may be beneficial for optimal edge deployment. Future research directions should investigate federated learning implementation for privacy-preserving multi-site training, integration with mobile diagnostic platforms for field deployment, hardware-aware neural architecture search for resource-constrained medical imaging, and cross-domain adaptation techniques to address demographic robustness challenges. The established framework provides a foundation for scalable, interpretable AI deployment in global health applications where technological constraints challenge conventional deep learning approaches whilst maintaining the clinical accuracy standards essential for patient safety.

This project has profoundly enhanced my understanding of the intersection between advanced machine learning techniques and practical healthcare deployment challenges. Through systematic experimentation with diverse architectures, I gained invaluable insights into the delicate balance between diagnostic accuracy and computational efficiency required for real-world medical applications. The quantisation process particularly demonstrated how theoretical deep learning advances must be carefully adapted for resource-constrained environments. Most significantly, this work reinforced that meaningful AI in healthcare extends beyond achieving benchmark performance metrics to creating deployable solutions that can genuinely impact underserved communities. The integration of explainability mechanisms through Grad-CAM highlighted the critical importance of physician trust and clinical interpretability in medical AI systems, lessons that will guide my future work in healthcare technology.

References

[1] United Nations Children's Fund (UNICEF), "Pneumonia," 2025. [Online]. Available: https://data.unicef.org/topic/child-health/pneumonia/.

[2] Y. Wang et al., "A multi-modal deep learning solution for precise pneumonia diagnosis: the PneumoFusion-Net model," Frontiers in Physiology, vol. 16, p. 1512835, Mar. 2025.

[3] A. Lamia and A. Fawaz, "Detection of Pneumonia Infection by Using Deep Learning on a Mobile Platform," Computational Intelligence and Neuroscience, vol. 2022, p. 7925668, Jul. 2022.

[4] V. Kottawar, N. Deshpande, R. Deshmukh, and A. Shinde, "Design of System for Detection of Pneumonia using Deep Learning," Frontiers in Health Informatics, vol. 13, no. 3, pp. 10778–10793, 2024.

[5] Y. Xie, B. Zhu, Y. Jiang, B. Zhao, and H. Yu, "Diagnosis of pneumonia from chest X-ray images using YOLO deep learning," Frontiers in Neurorobotics, vol. 19, p. 1576438, Apr. 2025.

[6] X. Wang et al., "ChestX-ray8: Hospital-scale Chest X-ray Database and Benchmarks on Weakly-Supervised Classification and Localization of Common Thorax Diseases," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR), 2017, pp. 3462–3471.

[7] Q. An, W. Chen, and W. Shao, "A Deep Convolutional Neural Network for Pneumonia Detection in X-ray Images with Attention Ensemble," Diagnostics, vol. 14, no. 4, p. 390, Feb. 2024.

[8] P. Rajpurkar et al., "CheXNet: Radiologist-Level Pneumonia Detection on Chest X-rays with Deep Learning," arXiv:1711.05225, Nov. 2017.

[9] C. Chen et al., "Deep Learning on Computational-Resource-Limited Platforms: A Survey," Mobile Information Systems, vol. 2020, p. 8454327, Mar. 2020.

[10] K. Bertels et al., "Quantum computing—A new scientific revolution in the making," arXiv:2406.02601, Jun. 2021, revised May 2024.

[11] N. Raina et al., "Progress in achieving SDG targets for mortality reduction among mothers, newborns, and children in the WHO South-East Asia Region," The Lancet Regional Health—Southeast Asia, vol. 18, p. 100307, Oct. 2023.

[12] D. Kermany, K. Zhang, and M. Goldbaum, "Labeled Optical Coherence Tomography (OCT) and Chest X-Ray Images for Classification," Mendeley Data, vol. 2, 2018, doi: 10.17632/rscbjbr9sj.2.

[13] S. Thakur et al., "Optimization of Deep Learning Models for Inference in Low Resource Environments," in Proc. IEEE Conf. Comput. Vis. Pattern Recognit. Workshops, 2025.

[14] A. Agbeyangi and H. Suleman, "Advances and challenges in low-resource-environment software systems: A survey," Informatics, vol. 11, no. 4, p. 90, Nov. 2024.

[15] M. R. Hasan, S. M. A. Ullah, and S. M. R. Islam, "Recent advancement of deep learning techniques for pneumonia prediction from chest X-ray image," Medical Reports, vol. 7, p. 100106, 2024.

[16] A. Mabrouk et al., "Pneumonia Detection on Chest X-ray Images Using Ensemble of Deep Convolutional Neural Networks," Applied Sciences, vol. 12, no. 13, p. 6448, 2022.

[17] R. Kundu, R. Das, Z. C. Alex, and S. S. R. M. P., "Pneumonia detection in chest X-ray images using an ensemble of deep learning models," PLOS ONE, vol. 16, no. 9, p. e0256630, 2021.

[18] M. R. Hasan and S. M. A. Ullah, "Enhancing Pneumonia Diagnosis: An Ensemble of Deep CNN Architectures for Accurate Chest X-Ray Image Analysis," in International Conference on Big Data, IoT and Machine Learning, 2023, pp. 255–269.

[19] A. Howard et al., "Searching for MobileNetV3," in 2019 IEEE/CVF International Conference on Computer Vision (ICCV), Seoul, Korea (South), 2019, pp. 1314-1324.

[20] M. S. Yaraziz, N. S. Safa, and M. A. Azad, "Edge computing in IoT for smart healthcare," Journal of Ambient Intelligence and Smart Environments, vol. 16, no. 4, pp. 409–438, 2024.

[21] Y. Xu et al., "Edge deep learning in computer vision and medical diagnostics: A comprehensive survey," Artificial Intelligence Review, vol. 58, no. 2, p. 93, 2025.

[22] S. Choudhary et al., "Edge AI Deploying Artificial Intelligence Models on Edge Devices for Real-Time Analytics," ITM Web of Conferences, vol. 76, p. 01009, 2025.

[23] S. S. Gill et al., "Edge AI: A Taxonomy, Systematic Review and Future Directions," arXiv:2407.04053, 2024.

[24] M. Larrazabal et al., "Gender imbalance in medical imaging datasets produces biased classifiers for computer-aided diagnosis," Proceedings of the National Academy of Sciences, vol. 117, no. 23, pp. 12592–12594, 2020.

[25] J. Ma, Y. Song, X. Tian, Y. Hua, R. Zhang, and J. Wu, "Survey on deep learning for pulmonary medical imaging," Frontiers of Medicine, vol. 14, no. 4, pp. 450–469, 2020.

[26] A. Rajkomar, M. Hardt, M. D. Howell, G. Corrado, and M. H. Chin, "Ensuring Fairness in Machine Learning to Advance Health Equity," Annals of Internal Medicine, vol. 169, no. 12, pp. 866–872, 2018.

[27] V. Chouhan et al., "A novel transfer learning based approach for pneumonia detection in chest X-ray images," Applied Sciences, vol. 10, no. 2, p. 559, Jan. 2020.

[28] M. S. Al Reshan, "Detection of Pneumonia from Chest X-ray Images Utilizing MobileNet Model," Healthcare, vol. 11, no. 11, p. 1561, 2023.

[29] X. Xu, "A systematic review: Deep learning-based methods for pneumonia region detection," arXiv:2408.13315, Aug. 2024.

[30] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018, pp. 7132-7141.

[31] M. Sandler et al., "MobileNetV2: Inverted residuals and linear bottlenecks," in Proc. IEEE Conf. Computer Vision and Pattern Recognition, 2018, pp. 4510-4520.

[32] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. Int. Conf. Machine Learning, 2019, pp. 6105-6114.

[33] R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in Proc. IEEE Int. Conf. Computer Vision, 2017, pp. 618-626.

[34] A. Manzoor et al., "Efficient computation of attention mechanisms in medical image analysis through resolution optimization," IEEE Trans. Medical Imaging, vol. 42, no. 3, pp. 847-856, 2023.

[35] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2019, pp. 2623–2631.

[36] A. Mazur, "Pneumonia Detection using VGG16," GitHub repository, 2020. [Online]. Available: https://github.com/alicjamazur/pneumonia-detection

[37] A. Kumar, "Pneumonia Detection Using InceptionV3," GitHub repository, 2021. [Online]. Available: https://github.com/Abhaykumar04/Pneumonia-Detection-Using-InceptionV3

[38] S. Wizfury, "Pneumonia Detection Using MobileNetV2," GitHub repository, 2021. [Online]. Available: https://github.com/wizfury/PNEUMONIA-DETECTION-USING-MOBILENETV2

[39] M. Matic, "Pneumonia Detection using ResNet," GitHub repository, 2020. [Online]. Available: https://github.com/Mili4Matic/pneumonia-resnet

[40] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

Appendix

AI Assisted Model Implementation Details

Prompt 1: Custom Architecture Design

I am developing a lightweight convolutional neural network for pneumonia detection in resource-constrained environments. I need to implement a custom architecture combining inverted residual blocks with squeeze-and-excitation mechanisms and multi-scale feature extraction. The model should maintain efficiency for edge deployment while achieving clinical-grade accuracy. Could you provide guidance on the optimal integration of SE blocks within inverted residuals and the implementation of multi-scale feature extraction blocks that capture 1×1, 3×3, and 5×5 features through parallel convolution branches?

Response:

Guidance

- Place the SE block **after the depthwise conv + BN + activation and before the pointwise projection** (MobileNetV3 pattern).
- Use a small expansion (×3–×4) and a small SE squeeze ratio (r = 0.125–0.25) for edge efficiency.
- Implement 5×5 cheaply: two stacked 3×3 separable/depthwise convs or a 3×3 with `dilation_rate=2`.
- Keep residual connections only when `stride==1` and `in_channels==out_channels`.
- Use separable/depthwise convs for multi-scale branches to save compute.

TensorFlow Keras implementation

python Copy code

```python
import tensorflow as tf
from tensorflow.keras import layers, Model, backend as K

def se_block(x, se_ratio=0.25):
    channels = K.int_shape(x)[-1]
    reduced = max(1, int(channels * se_ratio))
    se = layers.GlobalAveragePooling2D()(x)
    se = layers.Reshape((1,1,channels))(se)
    se = layers.Conv2D(reduced, 1, activation='relu', use_bias=True)(se)
    se = layers.Conv2D(channels, 1, activation='sigmoid', use_bias=True)(se)
    return layers.Multiply()([x, se])
```

Copy code

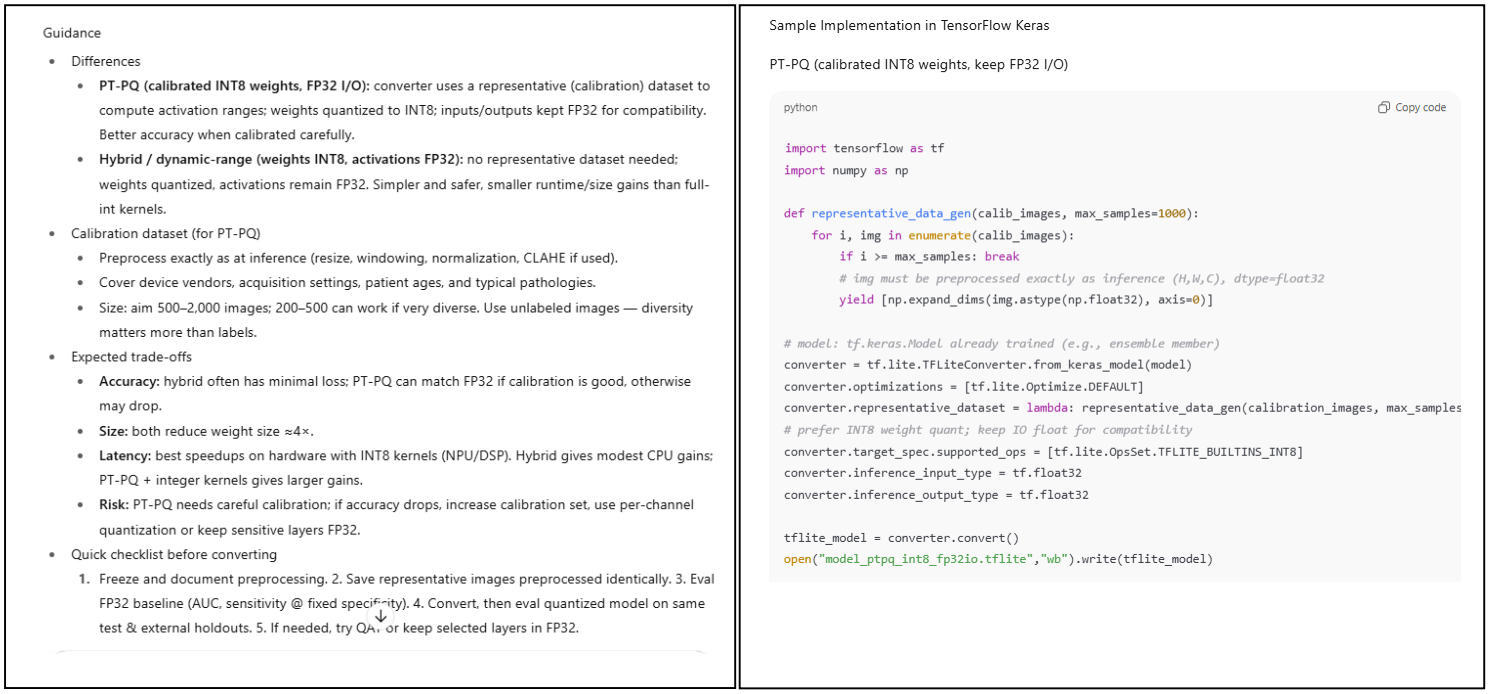
```python
def multi_scale_block(inputs, out_filters, activation=layers.ReLU):
    branch = max(8, out_filters // 3)
    b1 = layers.Conv2D(branch, 1, padding='same', use_bias=False)(inputs)
    b1 = layers.BatchNormalization()(b1); b1 = activation()(b1)
    b2 = layers.SeparableConv2D(branch, 3, padding='same', use_bias=False)(inputs)
    b2 = layers.BatchNormalization()(b2); b2 = activation()(b2)
    b3 = layers.SeparableConv2D(branch, 3, padding='same', use_bias=False)(inputs)
    b3 = layers.BatchNormalization()(b3); b3 = activation()(b3)
    b3 = layers.SeparableConv2D(branch, 3, padding='same', use_bias=False)(b3)
    b3 = layers.BatchNormalization()(b3); b3 = activation()(b3)
    x = layers.Concatenate()([b1, b2, b3])
    x = layers.Conv2D(out_filters, 1, padding='same', use_bias=False)(x)
    x = layers.BatchNormalization()(x); x = activation()(x)
    return x

def build_pneumonia_net(input_shape=(224,224,1), num_classes=1, width_mult=1.0, se_ratio=0.1875)
    inp = layers.Input(shape=input_shape)
    x = layers.Conv2D(int(16*width_mult), 3, strides=2, padding='same', use_bias=False)(inp)
    x = layers.BatchNormalization()(x); x = layers.ReLU()(x)
```

Figure A.1. Lightweight CNN with inverted residuals, SE blocks, and multi-scale extraction.

Prompt 2: Quantisation Strategy Optimisation

I am implementing two distinct quantisation strategies for medical imaging models targeting edge deployment. For my ensemble model on paediatric data, I need to implement post-training parameter quantisation (PT-PQ) with INT8 parameter quantisation while maintaining FP32 input/output conversion. 2nd: For my EfficientNetB4 model on a different dataset, I require hybrid dynamic range quantisation that quantises only weights to INT8 whilst preserving activations in FP32. Could you provide guidance on the implementation differences between these approaches, optimal calibration dataset selection for medical imaging, and expected performance trade-offs between PT-PQ and hybrid quantisation methods for convolutional neural networks in clinical applications?

Response:



Guidance

- Differences
  - **PT-PQ (calibrated INT8 weights, FP32 I/O):** converter uses a representative (calibration) dataset to compute activation ranges; weights quantized to INT8; inputs/outputs kept FP32 for compatibility. Better accuracy when calibrated carefully.
  - **Hybrid / dynamic-range (weights INT8, activations FP32):** no representative dataset needed; weights quantized, activations remain FP32. Simpler and safer, smaller runtime/size gains than full-int kernels.
- Calibration dataset (for PT-PQ)
  - Preprocess exactly as at inference (resize, windowing, normalization, CLAHE if used).
  - Cover device vendors, acquisition settings, patient ages, and typical pathologies.
  - Size: aim 500–2,000 images; 200–500 can work if very diverse. Use unlabeled images — diversity matters more than labels.
- Expected trade-offs
  - **Accuracy:** hybrid often has minimal loss; PT-PQ can match FP32 if calibration is good, otherwise may drop.
  - **Size:** both reduce weight size ≈4×.
  - **Latency:** best speedups on hardware with INT8 kernels (NPU/DSP). Hybrid gives modest CPU gains; PT-PQ + integer kernels gives larger gains.
  - **Risk:** PT-PQ needs careful calibration; if accuracy drops, increase calibration set, use per-channel quantization or keep sensitive layers FP32.
- Quick checklist before converting
  1. Freeze and document preprocessing. 2. Save representative images preprocessed identically. 3. Eval FP32 baseline (AUC, sensitivity @ fixed specificity). 4. Convert, then eval quantized model on same test & external holdouts. 5. If needed, try QAT or keep selected layers in FP32.

Sample Implementation in TensorFlow Keras

PT-PQ (calibrated INT8 weights, keep FP32 I/O)

python — Copy code

```python
import tensorflow as tf
import numpy as np

def representative_data_gen(calib_images, max_samples=1000):
    for i, img in enumerate(calib_images):
        if i >= max_samples: break
        # img must be preprocessed exactly as inference (H,W,C), dtype=float32
        yield [np.expand_dims(img.astype(np.float32), axis=0)]

# model: tf.keras.Model already trained (e.g., ensemble member)
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = lambda: representative_data_gen(calibration_images, max_samples
# prefer INT8 weight quant; keep IO float for compatibility
converter.target_spec.supported_ops = [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.float32
converter.inference_output_type = tf.float32

tflite_model = converter.convert()
open("model_ptpq_int8_fp32io.tflite","wb").write(tflite_model)
```

Figure A.2: PT-PQ vs hybrid quantisation strategies for edge medical models.

Prompt 3: Ensemble Meta-Learning Implementation

I am developing a Nuclear Ensemble architecture combining ResNet50, DenseNet121, InceptionV3, Xception, and MobileNetV2 for pneumonia detection. I need to implement a meta-learning framework with weighted voting optimisation and uncertainty quantification through Monte Carlo dropout. Could you provide guidance on implementing dual-stage prediction frameworks with automated hyperparameter optimisation for optimal model weighting, and how to integrate focal loss for handling severe class imbalance in medical datasets?

Response:

Thought for 4 seconds

Guidance

- **Dual-stage:** Stage-1 — train base models (ResNet50, DenseNet121, InceptionV3, Xception, MobileNetV2) with Dropout before the head for MC-dropout. Stage-2 — aggregate Stage-1 outputs with a learned meta-aggregator (stacking or constrained weighted-voting).
- **Weighted voting:** implement trainable non-negative weights via learnable logits + softmax (or find fixed weights with Optuna/grid search on validation AUC).
- **Uncertainty (MC-dropout):** run N stochastic forward passes per base (`training=True`), use mean for prediction and variance/entropy for epistemic uncertainty; combine across models.
- **HPO:** tune meta learning rate, meta size, dropout rates, MC samples, focal gamma/alpha, and ensemble priors with Optuna/TPE and stratified CV.
- **Focal loss:** use binary focal loss (gamma ~1–3, alpha for class balance) + stratified sampling/oversampling.
- **Practical:** freeze preprocessing, evaluate FP32 baselines (AUC, sensitivity @ specificity), generate validation folds for meta-training, re-evaluate after aggregation.

Sample Implementation in TensorFlow Keras

python Copy code

```python
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, Model, backend as K
import optuna
from sklearn.metrics import roc_auc_score

# focal loss
def focal_loss(gamma=2.0, alpha=0.25):
    def loss(y_true, y_pred):
        y_true = tf.cast(y_true, tf.float32)
        bce = tf.keras.losses.binary_crossentropy(y_true, y_pred)
        p_t = y_true * y_pred + (1 - y_true) * (1 - y_pred)
        alpha_t = y_true * alpha + (1 - y_true) * (1 - alpha)
        return tf.reduce_mean(alpha_t * tf.pow(1.0 - p_t, gamma) * bce)
    return loss

# base model factory with MC dropout
def build_base(name, input_shape=(224,224,3), dropout=0.3):
    cls = {
      'resnet50': tf.keras.applications.ResNet50,
      'densenet121': tf.keras.applications.DenseNet121,
      'inceptionv3': tf.keras.applications.InceptionV3,
      'xception': tf.keras.applications.Xception,
      'mobilenetv2': tf.keras.applications.MobileNetV2
    }[name]
    base = cls(include_top=False, weights='imagenet', input_shape=input_shape, pooling='avg')
    x = layers.Dropout(dropout)(base.output)
    logits = layers.Dense(1)(x)
    probs = layers.Activation('sigmoid')(logits)
    return Model(base.input, [logits, probs], name=f"base_{name}")

# MC-dropout inference
```

Figure A.3: Nuclear Ensemble with weighted voting, MC-dropout, and focal loss.

Prompt 4: Explainability Optimisation

I am integrating Grad-CAM explainability into my pneumonia detection models for clinical deployment. The challenge is achieving real-time explanation generation on resource-constrained hardware. I need to optimise the computational efficiency of gradient-weighted class activation mapping whilst maintaining explanation fidelity. How can I implement resolution optimisation strategies and target layer identification across diverse architectures to achieve significant computational speedup without compromising clinical interpretability?

Response:

Thought for 30s

Guidance

- **Target layer choice (fidelity vs speed):** use the **last high-level convolutional block before global pooling** (small spatial size → faster). Good defaults:
  - ResNet50: `conv5_block3_out`
  - DenseNet121: `conv5_block16_concat`
  - InceptionV3: `mixed10`
  - Xception: `block14_sepconv2_act`
  - MobileNetV2: `out_relu` (or `block_16_project`)

  Prefer the smallest feature map that still localizes lungs well; if heatmaps look coarse, back off one block earlier.
- **Resolution optimization:** compute CAM on the small target map (e.g., 7×7…14×14) and **bilinearly upsample** to the display size. If needed, **multilayer fusion** (e.g., average of CAMs from last and penultimate blocks) recovers detail with little cost (+1 backprop).
- **Runtime tips:**
  - Build a **grad-model** that outputs `[target_layer_activations, logits]` and backprop only to the target layer (no need to track earlier layers).
  - **tf.function**-compile the Grad-CAM step; reuse a single prebuilt grad-model.
  - Compute only the **predicted class logit** (or the clinical target label), not all classes.
  - **Batch CAMs** for multiple images at once.
  - Mixed precision (fp16) if supported; keep outputs/resizing in fp32 for numerical stability.
- **MC-Dropout compatibility:** run `training=True` for N stochastic passes, **average CAMs** to reduce variance; N=4–8 is often enough on edge.
- **Clinical guardrails:** standardize preprocessing; normalize CAMs per-image; check **sensitivity at fixed specificity** with and without CAM generation to firm no latency-induced timeouts; keep deterministic fallback (single pass).

Sample Implementation in TensorFlow Keras

python

Copy code

```python
import tensorflow as tf
from tensorflow.keras import Model

# Build grad-model once
def build_grad_model(model):
    target_layer = model.get_layer("block14_sepconv2_act").output
    return Model([model.inputs], [target_layer, model.outputs[0]])

grad_model = build_grad_model(xception_model)

# Grad-CAM step
@tf.function
def gradcam_step(x):
    with tf.GradientTape() as tape:
        conv_out, preds = grad_model(x, training=False)
        class_score = preds[:, 0]  # binary positive logit
    grads = tape.gradient(class_score, conv_out)
    weights = tf.reduce_mean(grads, axis=[1,2], keepdims=True)
    cam = tf.nn.relu(tf.reduce_sum(weights * conv_out, axis=-1, keepdims=True))
    cam_min, cam_max = tf.reduce_min(cam, [1,2,3], keepdims=True), tf.reduce_max(cam, [1,2,3], kee
    return (cam - cam_min) / (cam_max - cam_min + 1e-8)

# Upsample CAM to input size
def upsample_cam(cam_small, to_size):
    return tf.image.resize(cam_small, to_size, method="bilinear")

# Example usage
# cam_small = gradcam_step(x_batch)                    # (B, h, w, 1)
```

Figure A.4: Grad-CAM optimisation with resolution scaling and target layer selection.